

ملخص شامل لـ

CSS

دليل تعليمي مختصر ومنظم

Amine Mohamed

١٤ نوفمبر ٢٠٢٠

جدول المحتويات

1. مقدمة إلى CSS — ما هي CSS ولماذا نستخدمها؟

2. مقدمة إلى CSS — بنية CSS الأساسية (القاعدة والمُحدّد والخاصية والقيمة)

3. مقدمة إلى CSS — طرق ربط CSS بصفحات HTML (مُضمّن، داخلي، خارجي)

4. مقدمة إلى CSS — التعليقات في CSS

5. المحددات في CSS — المحددات الأساسية: العنصر، الفئة (Class)، الهوية (ID)

6. المحددات في CSS — محدّدات التجميع والترابط (Combinators)

7. المحددات في CSS — محدّدات السمات (Attribute Selectors)

8. المحددات في CSS — مفهوم الأولوية (Specificity) والتسلسل (Cascade)

9. الألوان، الخلفيات، والخطوط — التحكم بالألوان (الأسماء، الهيكس، RGB, HSL)

10. الألوان، الخلفيات، والخطوط — خصائص الخلفية (اللون، الصورة، التكرار، الموضع، الحجم)

11. الألوان، الخلفيات، والخطوط — التحكم بالخطوط: عائلة الخط (Font-family)، الحجم (Font-size)، الوزن (Font-weight)

12. الألوان، الخلفيات، والخطوط — خصائص النص: المحاذاة، التزيين، المسافات بين الحروف والكلمات

13. نموذج الصندوق (Box Model) — فهم نموذج الصندوق ومكوناته

14. نموذج الصندوق (Box Model) — الهامش الخارجي (Margin)

15. نموذج الصندوق (Box Model) — الحدود (Border)

16. نموذج الصندوق (Box Model) — الحشوة الداخلية (Padding)

17. نموذج الصندوق (Box Model) — المحتوى (Content)

18. خصائص العرض والتوضع (Display and Positioning) — خصائص العرض (Display): block, inline, inline-block, none

19. خصائص العرض والتوضع (Display and Positioning) — التوضع النسبي (Relative Positioning) والتوضع الثابت (Static Positioning)

20. خصائص العرض والتوضع (Display and Positioning) — التوضع المطلق (Absolute Positioning) والتوضع اللزج (Fixed/Sticky)

(Positioning)

21. خصائص العرض والتوضع (Display and Positioning) — التحكم بترتيب العناصر (Z-index)

22. التصميم المرن (Flexbox) — مقدمة إلى Flexbox ومفهوم المحور الرئيسي والفرعي

23. التصميم المرن (Flexbox) — خصائص الحاوية المرنة (Flex Container): الاتجاه، المحاذاة، التبرير

24. التصميم المرن (Flexbox) — خصائص العناصر المرنة (Flex Items): التمدد، الانكماش، القاعدة

25. التصميم المرن (Flexbox) — التفاف العناصر (Wrapping) وتوزيع المساحات

26. شبكة (CSS Grid) CSS — مقدمة إلى CSS Grid وتعريف الشبكة

27. شبكة (CSS Grid) CSS — تحديد الأعمدة والصفوف (grid-template-columns, grid-template-rows)

28. شبكة (CSS Grid) CSS — وضع العناصر داخل الشبكة (grid-column, grid-row)

29. شبكة (CSS Grid) CSS — مناطق الشبكة (Grid Areas) والفجوات (Gaps)

30. التصميم المتجاوب (Responsive Design) — مقدمة للتصميم المتجاوب وأهميته

31. التصميم المتجاوب (Responsive Design) — استعلامات الوسائط (Media Queries)

32. التصميم المتجاوب (Responsive Design) — وحدات القياس المرنة: em, rem, vw, vh

33. التصميم المتجاوب (Responsive Design) — الصور المتجاوبة (Responsive Images)

34. المحددات المتقدمة والفئات/العناصر الزائفة — المحددات الزائفة للفئة (Pseudo-classes): hover, focus, nth-child

35. المحددات المتقدمة والفئات/العناصر الزائفة — المحددات الزائفة للعنصر (Pseudo-elements): before, after, first-line

36. المحددات المتقدمة والفئات/العناصر الزائفة — محددات العلاقة (Combinators المتقدمة)

37. المحددات المتقدمة والفئات/العناصر الزائفة — محددات النفي والتطابق (not(), :is())

38. الانتقالات والتحويلات (Transitions and Transforms) — الانتقالات في CSS (Transitions): الخاصية، المدة، دالة التوقيت، التأخير

39. الانتقالات والتحويلات (Transitions and Transforms) — التحويلات ثنائية الأبعاد (2D Transforms): translate, rotate, scale, skew

40. الانتقالات والتحويلات (Transitions and Transforms) — التحويلات ثلاثية الأبعاد (3D Transforms): rotateX, rotateY, perspective

41. الانتقالات والتحويلات (Transitions and Transforms) — تطبيق الانتقالات والتحويلات لتأثيرات بصرية جذابة

42. المتغيرات والدوال في CSS — المتغيرات المخصصة (Custom Properties / CSS Variables) وكيفية تعريفها

43. المتغيرات والدوال في CSS — استخدام المتغيرات (var()) وتحديد نطاقها (Scope)

44. المتغيرات والدوال في CSS — الدوال الشائعة في CSS مثل calc()

45. المتغيرات والدوال في CSS — تنظيم الأنماط وإعادة استخدامها باستخدام المتغيرات

46. بناء مشاريع بسيطة وتطبيق المفاهيم — تطبيق المفاهيم: بناء صفحة هبوط بسيطة (Landing Page)

47. بناء مشاريع بسيطة وتطبيق المفاهيم — تصميم بطاقات المنتج أو الخدمات (Product/Service Cards)

48. بناء مشاريع بسيطة وتطبيق المفاهيم — بناء قائمة تنقل

49. بناء مشاريع بسيطة وتطبيق المفاهيم — مراجعة أفضل الممارسات
في كتابة CSS

مقدمة إلى CSS — ما هي CSS ولماذا نستخدمها؟

ما هي CSS ولماذا نستخدمها؟

في هذا الدرس، سنتعرف على لغة CSS وأهميتها الكبيرة في تصميم وتجميل صفحات الويب. ستفهم لماذا لا يمكنك الاستغناء عنها عند بناء أي موقع إلكتروني.

ما هي CSS؟

CSS هي اختصار لـ **Cascading Style Sheets**، وهي لغة تستخدم لوصف مظهر وتنسيق محتوى صفحات الويب المكتوبة بلغة HTML. إنها تفصل بين المحتوى (مثل النصوص والصور) وبين كيفية عرضه (مثل الألوان، الخطوط، المسافات، والتخطيط).

مثال عملي

```
h1 {  
  color: blue;  
  font-family: sans-serif;  
  text-align: center;  
}
```

شرح الكود

هذا الكود يحدد أن جميع عناوين المستوى الأول (**h1**) في صفحة الويب ستظهر باللون **الأزرق**، وباستخدام خط من عائلة الخطوط **sans-serif**، وستكون **متوسطة** في المنتصف. هذا يُظهر كيف تتحكم CSS في مظهر عنصر HTML محدد.

- **CSS** هي لغة أساسية لتنسيق وتصميم صفحات الويب وجعلها جذابة.

- تفصل **CSS** مظهر الموقع عن هيكله (HTML)، مما يسهل التعديل والصيانة.
- باستخدام **CSS**، يمكنك التحكم الكامل في الألوان، الخطوط، التخطيط، وتجربة المستخدم البصرية.

مقدمة إلى CSS — بنية CSS الأساسية

(القاعدة والمُحدّد والخاصية والقيمة)

بنية CSS الأساسية

لفهم كيفية عمل CSS، من المهم أن نتعرف على بنيتها الأساسية. تتكون كل "قاعدة" في CSS من أجزاء محددة تعمل معًا لتطبيق الأنماط على عناصر HTML في صفحة الويب.

المُحدّد (Selector)

المُحدّد هو الجزء الذي يخبر المتصفح أي عنصر أو مجموعة من العناصر في HTML يجب تطبيق الأنماط عليها. يمكن أن يكون اسم وسم (مثل `h1`)، أو فئة (`.class`)، أو معرّف (`#id`).

الخاصية (Property) والقيمة (Value)

بعد المُحدّد، تأتي كتلة الإعلان بين قوسين متعرجين `{ }`. داخل هذه الكتلة، نحدد الخاصية التي نريد تغييرها (مثل `color` لتغيير اللون)، والقيمة التي تحدد كيفية التغيير (مثل `blue` ليكون اللون أزرق). كل زوج من الخاصية والقيمة يُسمى "إعلان" وينتهي بفاصلة منقوطة `;`.

مثال توضيحي

```
h1 {  
  color: blue;  
  font-size: 24px;  
}
```

شرح الكود

في هذا المثال، `h1` هو المُحدّد الذي يستهدف جميع عناوين المستوى الأول. داخل القوسين المتعرجين، لدينا إعلانان: الأول هو `color: blue`؛ حيث `color` هي الخاصية و `blue` هي القيمة.

مما يجعل لون النص أزرق. الثاني هو `font-size: 24px`; حيث `font-size` هي الخاصية و `24px` هي القيمة، مما يحدد حجم الخط بـ 24 بكسل. هذه الأجزاء مجتمعة تشكل قاعدة CSS كاملة.

- تتكون قاعدة CSS من **مُحدِّد** وكتلة إعلانية `{ }`.
- **المُحدِّد** يشير إلى عناصر HTML التي سيتم تنسيقها.
- كل إعلان داخل الكتلة يتكون من **خاصية** و**قيمة**، تفصل بينهما نقطتان رأسيتان : وتنتهي بفاصلة منقوطة ;.

مقدمة إلى CSS — طرق ربط CSS بصفحات HTML (مُضمّن، داخلي، خارجي)

طرق ربط CSS بصفحات HTML

لجعل صفحات الويب جذابة ومنظمة، نحتاج لربط تنسيقات CSS بملفات HTML. يوفر CSS ثلاث طرق رئيسية لربطه بصفحة الويب، ولكل منها استخداماته الخاص ومميزاته التي يجب فهمها لإنشاء تصميمات فعالة.

طرق ربط CSS

هناك ثلاث طرق أساسية لربط تنسيقات CSS بصفحات HTML: **الربط المُضمّن (Inline)** الذي يوضع مباشرة في الوسوم، **الربط الداخلي (Internal)** الذي يوضع داخل وسم `<style>` في رأس الصفحة، و**الربط الخارجي (External)** وهو الأسلوب المفضل عبر ملف CSS منفصل يتم ربطه بصفحة HTML.

مثال عملي

```
<!DOCTYPE html>
<html lang="ar">
  <head>
    <meta charset="UTF-8">
    <title>ربط CSS</title>
    <link rel="stylesheet" href="styles.css">
    <style>
      h2 {
        color: purple;
        text-align: center;
      }
    </style>
  </head>
  <body>
    <p style="color: blue; font-size: 18px;">نص بتنسيق مُضمّن</p>
```

```
</h2>.عنوان بتنسيق داخلي</h2>
```

```
<div id="external-element">عنصر بتنسيق خارجي</div>  
</body>  
</html>
```

شرح الكود

يطبق الكود ثلاثة أنواع من الربط: الأول على الفقرة يجعل لونها أزرق وحجم خطها 18 بكسل (مُضمّن). الثاني يحدد لون ومركزية العنوان `h2` (داخلي). أما الثالث فيربط بملف `styles.css` خارجي لتنسيق العنصر ذي المعرّف `#external-element`، موفراً بذلك مرونة وسهولة في إدارة التنسيقات.

خلاصة الدرس

- **الربط الخارجي** هو الأفضل للمشاريع الكبيرة لأنه يوفر تنظيماً وسهولة في الصيانة.
- **الربط الداخلي** مفيد لتطبيق تنسيقات خاصة بصفحة واحدة فقط.
- **الربط المُضمّن** يُستخدم لتغييرات بسيطة ومحددة جداً، ولكن يجب تجنبه قدر الإمكان للحفاظ على فصل المحتوى عن التنسيق.

مقدمة إلى CSS — التعليقات في CSS

التعليقات في CSS

التعليقات (Comments) هي أجزاء من الكود لا يتم تنفيذها بواسطة المتصفح، ولكنها مفيدة جدًا للمطورين. تساعدك التعليقات في توثيق الأجزاء المختلفة من ورقة الأنماط الخاصة بك، مما يسهل فهمها وصيانتها لاحقًا.

ما هي التعليقات؟

التعليقات في CSS هي ملاحظات تكتبها داخل ملف CSS الخاص بك. تبدأ التعليقات دائمًا بـ `/*` وتنتهي بـ `*/`. يقوم المتصفح بتجاهل أي شيء يقع بين هذين الرمزین ولا يؤثر على كيفية عرض الصفحة.

مثال عملي

```
/* هذا هو تعليق يصف النمط التالي */
body {
  background-color: lightblue; /* تعليق على السطر لنوع الخلفية */
  color: #333;
}

h1 {
  font-family: Arial, sans-serif;
  /* يمكن أن يمتد التعليق */
  /* على عدة أسطر لشرح مفصل */
  text-align: center;
}
```

شرح الكود

في هذا المثال، تظهر كيفية استخدام التعليقات في CSS. التعليق الأول يشرح الغرض العام للنمط الذي يليه للعنصر `body`. التعليق الثاني يوضح تفصيلًا لخاصية معينة على نفس السطر. أما التعليق الأخير، فيوضح أن التعليقات يمكن أن تمتد على عدة أسطر لشرح أطول وأكثر تفصيلًا. المتصفح يتجاهل كل هذه التعليقات تمامًا ولا تؤثر على عرض الصفحة.

- تُستخدم التعليقات (`/* ... */`) لتوثيق الكود وشرحه.
- يقوم المتصفح بتجاهل التعليقات ولا تؤثر على تصميم الصفحة أو أدائها.
- تساعد التعليقات في تحسين قابلية قراءة الكود وتسهيل صيانته وتطويره لاحقًا.

المحددات في CSS — المحددات الأساسية:

العنصر، الفئة (Class)، الهوية (ID)

المحددات الأساسية في CSS: العنصر، الفئة (Class)، الهوية (ID)

المحددات في CSS هي الأدوات التي تمكنك من استهداف عناصر HTML معينة لتطبيق الأنماط والتصميم عليها. فهم هذه المحددات الأساسية ضروري لأي شخص يتعلم CSS، لأنها الأساس لتصميم صفحات ويب متقنة ومتحكم بها. سنتناول هنا ثلاثة أنواع رئيسية: محدد العنصر، محدد الفئة، ومحدد الهوية.

مفهوم المحددات الأساسية

تتيح لك المحددات تطبيق الأنماط بدقة. كل نوع من المحددات له طريقة استخدام وهدف مختلف:

- **محدد العنصر (Element Selector):** يستهدف جميع العناصر من نوع HTML معين، مثل كل الفقرات (p) أو كل العناوين (h1).
- **محدد الفئة (Class Selector):** يستهدف أي عنصر يحمل اسم فئة (class) معينة. يمكنك تطبيق نفس الفئة على عدة عناصر مختلفة. يتم تمثيله بنقطة (.) متبوعة باسم الفئة.
- **محدد الهوية (ID Selector):** يستهدف عنصرًا واحدًا وفريدًا في الصفحة يحمل هوية (id) معينة. يجب أن تكون الهوية فريدة لكل صفحة HTML. يتم تمثيله بعلامة الهاش (#) متبوعة باسم الهوية.

مثال عملي

```
p {  
  color: blue;  
  font-family: sans-serif;  
}  
  
.highlight {
```



```
background-color: yellow;
font-weight: bold;
}

#main-header {
text-align: center;
font-size: 2.5em;
color: #333;
}
```

شرح الكود

في هذا المثال، تم تطبيق اللون الأزرق ونوع خط `sans-serif` على جميع الفقرات (p) في الصفحة. أي عنصر يحمل الفئة `highlight` سيحصل على خلفية صفراء وخط عريض. وأخيرًا، العنوان الرئيسي الذي يحمل الهوية `main-header` سيتم توسيطه، وسيكون حجم خطه كبيرًا، ولونه رماديًا داكنًا.

خلاصة الدرس

- **محدد العنصر:** يطبق الأنماط على جميع عناصر HTML من نفس النوع.
- **محدد الفئة (.) :** يتيح لك تطبيق نفس الأنماط على عدة عناصر مختلفة.
- **محدد الهوية (#) :** يستخدم لاستهداف عنصر واحد وفريد فقط في صفحة HTML.

المحددات في CSS — محددات التجميع والترابط (Combinators)

المحددات في CSS: محددات التجميع والترابط (Combinators)

محددات التجميع والترابط (Combinators) في CSS هي أدوات قوية تسمح لك باختيار العناصر بناءً على علاقتها ببعضها البعض في بنية HTML. هي مفتاح لتطبيق الأنماط بدقة عالية وتجنب تضارب الأنماط غير المرغوب فيه.

ما هي محددات التجميع والترابط؟

هي رموز خاصة توضع بين محددين لتحديد علاقة الأبوة، البنية، أو الجوار بين العناصر. هذه المحددات تتيح لك استهداف عناصر معينة داخل هيكل معقدة:

- **المحدد السليل (Descendant Selector):** يحدد عنصرًا داخل عنصر آخر (مسافة).
- **المحدد الابن المباشر (Child Selector):** يحدد عنصرًا هو ابن مباشر لعنصر آخر (<).
- **المحدد الشقيق المجاور (Adjacent Sibling Selector):** يحدد عنصرًا يأتي مباشرة بعد عنصر آخر (+).
- **المحدد الشقيق العام (General Sibling Selector):** يحدد عنصرًا يأتي بعد عنصر آخر في نفس المستوى (~).

مثال على الكود

```
<div class="container">
  <p>هذه الفقرة 1 داخل الحاوية</p>
  <span>هذا سبان ابن مباشر</span>
  <p>هذه الفقرة 2 داخل الحاوية</p>
</div>

<h2>عنوان المقال</h2>
```

</p> هذه الفقرة تأتي مباشرة بعد العنوان<p>

```
<style>
/* المحدد السليل */
.container p {
    color: blue;
}

/* المحدد الابن المباشر */
.container > span {
    font-weight: bold;
}

/* المحدد الشقيق المجاور */
h2 + p {
    background-color: yellow;
}
</style>
```

شرح الكود

في المثال أعلاه، المحدد `container p` يطبق اللون الأزرق على جميع الفقرات (`<p>`) الموجودة داخل العنصر الذي يحمل الفئة `container`، بغض النظر عن مستوى تعشيشها. أما المحدد `container > span`، فيجعل خط العنصر `<span>` الابن المباشر للعنصر `container` عريضًا. وأخيرًا، `h2 + p` يلون خلفية الفقرة (`<p>`) التي تأتي مباشرة بعد عنصر العنوان `<h2>` باللون الأصفر.

نقاط أساسية

- تسمح محددات التجميع باستهداف العناصر بدقة بناءً على علاقاتها الهيكلية.
- تساعد على إنشاء أنماط CSS أكثر تنظيماً وقابلية للصيانة.
- فهم هذه المحددات ضروري لكتابة كود CSS فعال ومتقدم.

المحددات في CSS — محددات السمات (Attribute Selectors)

محددات السمات (Attribute Selectors)

في CSS، يتيح لك محددات السمات اختيار العناصر بناءً على وجود سمة HTML معينة أو قيمة تلك السمة. هذه الطريقة توفر مرونة كبيرة في استهداف العناصر لتطبيق تنسيقات محددة، وتتجاوز مجرد اختيار العناصر باسم الوسم أو الفئة.

ما هي محددات السمات؟

محددات السمات تسمح لك بتنسيق عناصر HTML بناءً على خصائصها (السمات) وقيمها، بدلاً من الاعتماد على الفئات (classes) أو المعرفات (IDs) فقط. يتم تمثيلها باستخدام الأقواس المعقوفة [].

مثال عملي

```
<!-- HTML -->
<input type="text" placeholder="اسم المستخدم">
<a href="https://example.com/secure">رابط آمن</a>
<a href="http://example.org/insecure">رابط غير آمن</a>

/* CSS */
input[type="text"] {
  border: 2px solid #007bff;
  padding: 8px;
}

a[href^="https"] {
  color: green;
  text-decoration: none;
}
```

شرح الكود

في هذا المثال، المحدد `input[type="text"]` يستهدف أي عنصر `<input>` يمتلك السمة `type` وقيمته `text`، مطبقًا عليه حدودًا زرقاء. أما المحدد `a[href^="https"]` فيستهدف أي عنصر `<a>` (رابط) تبدأ قيمة السمة `href` الخاصة به بـ `"https"`، مما يجعله يظهر باللون الأخضر وبدون خط سفلي.

- تستخدم محددات السمات `[]` لاستهداف العناصر بناءً على سماتها وقيمها.
- يمكنك مطابقة السمات بالضبط، أو مطابقة القيم التي تبدأ بـ `^=`، أو تنتهي بـ `$=`، أو تحتوي على `*=` نص معين.
- تساعد محددات السمات في تطبيق تنسيقات دقيقة ومرنة دون الحاجة لتغيير هيكل HTML بإضافة فئات إضافية.

المحددات في CSS — مفهوم الأولوية

(Specificity) والتسلسل (Cascade)

المحددات في CSS: مفهوم الأولوية (Specificity) والتسلسل (Cascade)

عندما تتعارض عدة قواعد CSS لتطبيق تنسيق على العنصر نفسه، فإن المتصفح يحتاج إلى طريقة لتحديد القاعدة التي ستفوز. هنا يأتي دور مفهومي **الأولوية (Specificity)** و**التسلسل (Cascade)**، وهما أساسيان لفهم كيفية عمل CSS.

الأولوية (Specificity)

الأولوية هي مقياس لتحديد مدى قوة أو "أهمية" محدد CSS. كل محدد يحصل على "درجة" معينة بناءً على نوعه (مثل: ID، Class، Element). المحدد ذو الدرجة الأعلى يفوز عند التعارض.

التسلسل (Cascade)

التسلسل هو الآلية التي يتبعها المتصفح لتجميع وتطبيق الأنماط من مصادر مختلفة (أنماط المتصفح الافتراضية، أنماط المستخدم، أنماط المطور). تعتمد عملية التسلسل على عدة عوامل، منها: ترتيب القواعد، أهمية القاعدة (مثل **important!**)، وأخيراً، الأولوية (Specificity).

مثال عملي

```
<!DOCTYPE html>
<html lang="ar">
  <head>
    <meta charset="UTF-8">
    <title>مثال الأولوية</title>
    <style>
      /* قاعدة 1: أولوية منخفضة (عنصر) */
      p {
```

```

        color: blue;
    }

    /* قاعدة 2: أولوية متوسطة (فئة) */
    .my-text {
        color: green;
    }

    /* قاعدة 3: أولوية عالية (معرف) */
    #special-paragraph {
        color: purple;
    }
</style>
</head>
<body>
    <p id="special-paragraph" class="my-text">هذه الفقرة ستكون
        باللون البنفسجي.</p>
</body>
</html>

```

شرح الكود

في المثال أعلاه، لدينا فقرة واحدة تستهدفها ثلاث قواعد CSS مختلفة لتغيير لون النص. على الرغم من أن الفقرة لها تنسيق افتراضي (p) وتنسيق فئة (.my-text)، إلا أن اللون النهائي الذي سيظهر هو البنفسجي (purple). هذا لأن المحدد ذو المعرف (#special-paragraph) يمتلك أعلى درجة أولوية (Specificity) بين جميع المحددات الأخرى، وبالتالي يتغلب على قواعد اللون الأزرق والأخضر.

نقاط رئيسية

- **الأولوية (Specificity):** تحدد مدى قوة المحددات؛ المحددات الأكثر تحديدًا (مثل IDs) لها أولوية أعلى من الأقل تحديدًا (مثل Classes والعناصر).
- **التسلسل (Cascade):** هي العملية التي يتبعها المتصفح لتحديد أي قاعدة CSS سيطبقها من بين المصادر والقواعد المتعارضة.
- فهم هذين المفهومين ضروري للتحكم الدقيق في تصميم صفحات الويب وتجنب السلوكيات غير المتوقعة في CSS.

الألوان، الخلفيات، والخطوط — التحكم بالألوان (الأسماء، الهيكس، RGB، HSL)

التحكم بالألوان في CSS (الأسماء، الهيكس، RGB، HSL)

الألوان أساسية لتصميم الويب ومنح صفحاتك مظهرًا جذابًا. في CSS، يمكنك تحديد الألوان بعدة طرق مرنة تتيح لك التحكم الدقيق في المظهر البصري للعناصر.

طرق تعريف الألوان

في CSS، يمكنك تحديد الألوان باستخدام عدة طرق شائعة. يمكنك استخدام **أسماء الألوان** المباشرة مثل `red` أو `blue`. لتحديد أكثر دقة، يمكنك استخدام **قيم الهيكس (Hexadecimal)** التي تبدأ بعلامة `#` تليها ستة أرقام وحروف (مثل `#FF0000`). هناك أيضًا **دالة RGB** التي تحدد اللون بدمج قيم الأحمر والأخضر والأزرق (`rgb(255, 0, 0)`)، ويمكن إضافة الشفافية باستخدام `rgba()`. وأخيرًا، **دالة HSL** التي تستخدم تدرج اللون والتشبع والسطوع (`hsl(0, 100%, 50%)`)، ويمكن أيضًا إضافة الشفافية باستخدام `hsla()`.

مثال على الكود

```
.text-color-name {
  color: dodgerblue;
}

.background-hex {
  background-color: #FF5733;
}

.text-rgb {
  color: rgb(30, 144, 255);
}

.background-hsl {
```



```
background-color: hsl(120, 70%, 50%);  
}
```

شرح الكود

يوضح الكود أعلاه أربع طرق مختلفة لتحديد الألوان: يُعيّن `text-color-name` لون نص باستخدام اسم اللون المباشر `dodgerblue`. يُعيّن `background-hex` لون الخلفية باستخدام قيمة هيكس `#FF5733`. يُعيّن `text-rgb` لون النص باستخدام دالة **RGB** بقيم الأحمر والأخضر والأزرق. يُعيّن `background-hsl` لون الخلفية باستخدام دالة **HSL**، حيث تحدد القيم تدرج اللون والتشبع والسطوع على الترتيب.

نقاط أساسية

- تتيح لك CSS تحديد الألوان بطرق متنوعة (الأسماء، الهيكس، HSL، RGB) للتحكم الكامل بالمظهر البصري.
- لكل طريقة مميزات؛ فالأسماء بسيطة، والهيكل و RGB دقيقة، و HSL سهلة الاستخدام للمصممين.
- يمكنك إضافة الشفافية للألوان باستخدام دالتي `rgba()` و `hsla()` لتأثيرات بصرية متنوعة.

الألوان، الخلفيات، والخطوط — خصائص الخلفية (اللون، الصورة، التكرار، الموضع، الحجم)

خصائص الخلفية في CSS

تعلم كيفية التحكم بمظهر خلفية عناصر HTML باستخدام خصائص CSS المختلفة، من تحديد الألوان وصولاً إلى استخدام الصور المعقدة. هذه الخصائص تتيح لك إضافة جمال وتصاميم جذابة لصفحات الويب الخاصة بك.

خصائص الخلفية الأساسية

تتيح لك CSS تحديد **لون الخلفية** ([background-color](#))، استخدام **صورة خلفية** ([background-image](#))، التحكم في كيفية **تكرار هذه الصورة** ([background-repeat](#))، تحديد **موضعها** داخل العنصر ([background-position](#))، وأخيراً التحكم في **حجمها** ([background-size](#)) لتناسب تصميمك.

مثال عملي

```
.my-element-background {
  background-color: #e0ffff;
  background-image:
    url('https://via.placeholder.com/150/e0ffff/000000?
    text=Pattern');
  background-repeat: no-repeat;
  background-position: center center;
  background-size: 80px;
  height: 200px;
  border: 1px solid #ccc;
}
```

شرح الكود

الكود أعلاه يقوم بتطبيق الخصائص التالية على عنصر يحمل الفئة `my-element-background`:

- `background-color`: يحدد لون أزرق سماوي فاتح كخلفية للعنصر.
- `background-image`: يستخدم صورة خارجية (مثال Placeholder) كخلفية.
- `background-repeat`: يمنع تكرار الصورة، فتظهر مرة واحدة فقط.
- `background-position`: يضع الصورة في منتصف العنصر تمامًا (أفقيًا وعموديًا).
- `background-size`: يحدد حجم الصورة لتكون 80 بكسل فقط.

خلاصة الدرس

- تتحكم خصائص الخلفية في المظهر الجمالي للعناصر، مما يجعلها أكثر جاذبية.
- يمكن دمج الألوان والصور والتحكم الدقيق في موضعها وتكرارها وحجمها.
- فهم هذه الخصائص أساسي لتصميم واجهات مستخدم احترافية ومرنة.

الألوان، الخلفيات، والخطوط — التحكم بالخطوط: عائلة الخط (Font-family)، الحجم (Font-size)، الوزن (Font-weight)

التحكم بالخطوط: عائلة الخط (Font-family)، الحجم (Font-size)، الوزن (Font-weight)

تُعد الخطوط جزءًا أساسيًا من تصميم أي موقع إلكتروني، فهي تؤثر بشكل مباشر على سهولة القراءة وجمالية المحتوى. في هذا الدرس، سنتعلم كيف تتحكم CSS في مظهر الخطوط باستخدام خصائص مهمة.

مفاهيم أساسية للتحكم بالخطوط

تتيح لك CSS تحديد عائلة الخط (Font-family) لتغيير نوع الخط، وحجم الخط (Font-size) للتحكم في مقاسه، ووزن الخط (Font-weight) لتحديد درجة سماكته (جريء أو عادي).

مثال عملي

```
.my-text {
  font-family: 'Amiri', serif;
  font-size: 20px;
  font-weight: bold;
}
```

شرح الكود

في هذا المثال، قمنا بتطبيق تنسيقات على عنصر له الفئة `.my-text`. خاصية `font-family` تحدد أن الخط هو "Amiri" وإذا لم يكن متاحًا، يتم استخدام أي خط من نوع "serif". تحدد `font-size` حجم الخط بـ 20 بكسل، بينما تجعل `font-weight` الخط سميكًا (bold).

- استخدم **font-family** لتحديد نمط الخط الرئيسي والخطوط البديلة.
- تحكم في قابلية القراءة باستخدام **font-size** لضبط حجم النص.
- أضف التأكيد على النص باستخدام **font-weight** لجعله سميكًا أو رقيقًا.

الألوان، الخلفيات، والخطوط — خصائص النص: المحاذاة، التزيين، المسافات بين الحروف والكلمات

خصائص النص: المحاذاة، التزيين، المسافات بين الحروف والكلمات

في CSS، لدينا القدرة على التحكم بشكل دقيق في مظهر النصوص لتناسب تصميمنا. سنتعلم هنا كيف نتحكم في محاذاة النص، إضافة تأثيرات تزيينية، وتحديد المسافات بين الحروف والكلمات، مما يعطينا مرونة كبيرة في تنظيم المحتوى.

محاذاة النص (Text Alignment)

تستخدم خاصية `text-align` لتحديد كيفية محاذاة النص داخل العنصر، مثل جعله في المنتصف، اليمين، اليسار، أو ضبطه (justify) لتوزيع الكلمات بشكل متساوٍ على طول السطر.

تزيين النص (Text Decoration)

خاصية `text-decoration` تسمح لنا بإضافة خطوط تحت النص (`underline`)، فوقه (`overline`)، أو في منتصفه (`line-through`)، أو إزالة أي تزيين موجود (`none`).

المسافات بين الحروف والكلمات (Letter and Word Spacing)

للتحكم في المسافات الأفقية: `letter-spacing` تحدد المسافة بين الحروف، و `word-spacing` تحدد المسافة بين الكلمات، مما يؤثر على قابلية النص للقراءة وجمالية التصميم.

مثال الكود

```
.my-styled-text {
    color: #2c3e50;
    font-family: 'Amiri', serif;
    text-align: justify;
    text-decoration: underline;
    letter-spacing: 0.5px;
    word-spacing: 2px;
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء فئة CSS باسم `.my-styled-text`. تحدد الخاصية `color` لون النص. `font-family` تختار نوع الخط المستخدم. `text-align: justify` تجعل النص محاذاة لكلا الطرفين. `text-decoration: underline` تضيف خطاً تحت النص. `letter-spacing: 0.5px` تزيد المسافة بين كل حرف وآخر بمقدار نصف بكسل. `word-spacing: 2px` تزيد المسافة بين الكلمات بمقدار 2 بكسل.

- تسمح خصائص النص بالتحكم الكامل في **مظهر النص وجمالياته** على الصفحة.
- `letter-spacing`، `text-decoration`، `text-align`، و `word-spacing` هي أدوات أساسية لتحسين **قابلية قراءة النص وتصميمه**.
- فهم هذه الخصائص يمكّنك من إنشاء **تجارب مستخدم بصرية أفضل** وتنسيق المحتوى بفعالية.

نموذج الصندوق (Box Model) — فهم نموذج الصندوق ومكوناته

فهم نموذج الصندوق ومكوناته

في عالم تصميم الويب، كل عنصر HTML تراه على الشاشة يُعامل كصندوق مستطيل. نموذج الصندوق (Box Model) في CSS هو مفهوم أساسي يشرح كيف تتكون هذه الصناديق وكيف يمكنك التحكم في أبعادها ومساحاتها وموضعها بالنسبة للعناصر الأخرى.

نموذج الصندوق (The Box Model)

يتكون كل صندوق من أربعة أجزاء رئيسية متحدة المركز: **المحتوى (Content)**، **الحشوة (Padding)**، **الإطار (Border)**، و**الهامش (Margin)**. فهم هذه الأجزاء يسمح لك بالتحكم الدقيق في تصميم وتخطيط العناصر على صفحتك.

مثال عملي

```
.my-box {  
  width: 200px;  
  height: 100px;  
  padding: 20px;  
  border: 5px solid #336699;  
  margin: 15px;  
  background-color: #e0f2f7;  
  text-align: center;  
  line-height: 100px;  
}
```

شرح الكود

الكود أعلاه يحدد صندوقاً بعرض 200 بكسل وارتفاع 100 بكسل. يتم إضافة حشوة داخلية (padding) بمقدار 20 بكسل حول المحتوى، مما يخلق مسافة بين المحتوى والإطار. بعد ذلك، يظهر إطار (border) سمكه 5 بكسل ولونه أزرق غامق. أخيراً، يُضاف هامش خارجي (margin) بمقدار 15 بكسل حول الصندوق، دافعاً إياه بعيداً عن العناصر المحيطة به.

خلاصة الفكرة

- **المحتوى (Content):** المساحة التي يعرض فيها المحتوى الفعلي (نصوص، صور).
- **الحشوة (Padding):** مساحة شفافة بين المحتوى والإطار، تتحكم في المسافة الداخلية.
- **الإطار (Border):** خط يحيط بالصندوق، بين الحشوة والهامش.
- **الهامش (Margin):** مساحة شفافة خارج الإطار، تتحكم في المسافة بين الصندوق والعناصر الأخرى.

نموذج الصندوق (Box Model) — الهامش الخارجي (Margin)

الهامش الخارجي (Margin)

في نموذج الصندوق (Box Model) في CSS، الهامش الخارجي هو أحد المكونات الأساسية التي تتحكم في المسافة بين العناصر. يساعدنا فهم الهامش الخارجي في ترتيب عناصر صفحة الويب بشكل فعال ومنظم، مما يضمن تباعدًا مناسبًا ومرئيًا.

ما هو الهامش الخارجي؟

الهامش الخارجي (Margin) هو مساحة شفافة تحيط بالحد (border) الخاص بالعنصر، وتقع خارجه. وظيفته الأساسية هي إبعاد العنصر عن العناصر الأخرى المحيطة به، مما يخلق فجوة أو مسافة حوله ويمنعها من الالتصاق ببعضها.

مثال على الهامش الخارجي

```
.my-box {  
  width: 150px;  
  height: 100px;  
  background-color: #f0f8ff;  
  border: 1px solid #0056b3;  
  margin: 20px;  
}
```

شرح الكود

في هذا المثال، قمنا بتحديد الهامش الخارجي للعنصر ذو الفئة `my-box` بقيمة 20 بكسل. هذا يعني أن العنصر سيحصل على مسافة 20 بكسل من الفراغ الشفاف حوله من جميع الجهات (الأعلى، الأسفل، اليمين، واليسار)، مما يدفعه بعيداً عن أي عناصر أخرى مجاورة ويخلق تباعدًا متساويًا.

- **يخلق مسافة خارج حدود العنصر:** الهامش الخارجي هو الفراغ المحيط بالعنصر من الخارج، بعد الحد مباشرة.
- **يدفع العناصر الأخرى بعيداً:** وظيفته الأساسية هي فصل العناصر عن بعضها البعض لضمان وضوح التصميم.
- **يمكن تحديده لجميع الجهات أو لجهة محددة:** يمكن تطبيق الهامش على جميع الجهات دفعة واحدة (`margin: 20px;`) أو بشكل منفصل (`margin-top, margin-right, margin-` `bottom, margin-left`).

نموذج الصندوق (Box Model) — الحدود (Border)

نموذج الصندوق: الحدود (Border)

في CSS، الحدود (Border) هي الخط الذي يحيط بالمحتوى والهوامش الداخلي (padding) لأي عنصر HTML. إنها جزء أساسي من نموذج الصندوق وتساعد على تحديد المساحة المرئية للعنصر وتمييزه عن العناصر الأخرى.

مفهوم الحدود

الحدود هي الطبقة الرابعة في نموذج الصندوق، وتقع بين الهامش الداخلي (padding) والهامش الخارجي (margin). يمكنك التحكم في سمكها، ونمطها (مثل متصل، متقطع، منقط)، ولونها، لإضافة لمسة جمالية أو تنظيمية لعناصر صفحتك.

مثال الكود

```
.my-box {  
    width: 200px;  
    height: 100px;  
    border-width: 3px;  
    border-style: solid;  
    border-color: #3498db;  
    /* أو يمكنك استخدام الاختصار */  
    /* border: 3px solid #3498db; */  
    padding: 10px;  
}
```

شرح الكود

في هذا المثال، قمنا بتطبيق حدود على عنصر يحمل الفئة `my-box`.. الخاصية `border-width` تحدد سمك الحدود بـ 3 بكسل. الخاصية `border-style` تحدد أن النمط سيكون `solid` (خط متصل). أما `border-color` فتحدد اللون الأزرق للحدود. يمكن دمج كل هذه الخصائص في خاصية واحدة مختصرة هي `border`، كما هو موضح في السطر المُعلق.

- **الحدود (Border)** هي الخطوط التي تحيط بالمحتوى والهامش الداخلي للعنصر.
- يمكن التحكم في سمك الحدود (`border-width`)، ونمطها (`border-style`)، ولونها (`border-color`).
- لتبسيط الكود، يمكنك استخدام الخاصية المختصرة `border` لتعيين كل هذه القيم دفعة واحدة.

نموذج الصندوق (Box Model) — الحشوة الداخلية (Padding)

الحشوة الداخلية (Padding) في CSS

في نموذج الصندوق (Box Model) في CSS، تُعد الحشوة الداخلية (**Padding**) جزءًا أساسيًا لتحديد المساحات. إنها المسافة الفارغة التي تضاف حول محتوى العنصر، وتفصله عن حدوده (Border).

ما هي الحشوة الداخلية (Padding)؟

الحشوة الداخلية هي المساحة الفارغة التي تضاف حول محتوى العنصر (مثل النص أو الصور) ولكنها تظل داخل حدود العنصر نفسه. تساعد هذه المساحة على "تنفس" المحتوى وتمنح التصميم مظهرًا أكثر تنظيمًا وجمالية.

مثال عملي

```
.my-element {
    background-color: #f0f8ff; /* لون خلفية فاتح */
    border: 1px solid #add8e6; /* حدود رقيقة */
    padding: 15px; /* حشوة داخلية متساوية من كل الجهات */
}

.another-element {
    background-color: #f0f8ff;
    border: 1px solid #add8e6;
    /* حشوة مختلفة لكل جهة: أعلى، يمين، أسفل، يسار */
    padding: 10px 20px 15px 25px;
}
```

شرح الكود

في المثال الأول، العنصر `my-element` يمتلك حشوة داخلية بقيمة **15 بكسل** مطبقة بالتساوي على جميع الجهات (أعلى، يمين، أسفل، يسار). هذا يخلق مساحة 15 بكسل بين المحتوى وحدود العنصر.

في المثال الثاني، العنصر `another-element` يستخدم خاصية `padding` كاختصار لتحديد قيم حشوة مختلفة لكل جهة: **10 بكسل** للأعلى، **20 بكسل** لليمين، **15 بكسل** للأسفل، و **25 بكسل** لليسار.

- **الحشوة الداخلية (Padding)** هي المساحة بين محتوى العنصر وحدوده.
- تُستخدم لإنشاء مساحة بيضاء حول المحتوى، لتحسين قراءة وتصميم الصفحة.
- يمكن تطبيقها على جميع الجهات بالتساوي، أو لكل جهة على حدة باستخدام الاختصارات.

نموذج الصندوق (Box Model) — المحتوى (Content)

المحتوى (Content)

في نموذج الصندوق (Box Model) الخاص بـ CSS، يُعامل كل عنصر HTML كصندوق مستطيل له أجزاء مختلفة. الجزء الأساسي والأكثر أهمية هو **المحتوى**، وهو المكان الذي يظهر فيه المحتوى الفعلي للعنصر.

ما هو المحتوى؟

المحتوى هو المنطقة الداخلية في الصندوق حيث تظهر النصوص، الصور، الروابط، أو أي محتوى آخر تضعه داخل العنصر. إنه لبّ العنصر، وتُحدد أبعاده الرئيسية عادةً باستخدام خصائص `width` (العرض) و `height` (الارتفاع).

مثال الكود

```
.my-content-box {
    width: 200px;
    height: 100px;
    background-color: #f0f8ff; /* AliceBlue */
    border: 1px solid #add8e6; /* LightBlue */
    padding: 15px;
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء فئة CSS تُسمى `.my-content-box`. الخاصية `width: 200px` تحدد أن عرض منطقة المحتوى سيكون 200 بكسل، بينما `height: 100px` تحدد أن ارتفاعها سيكون 100 بكسل. اللون الأزرق الفاتح هو خلفية هذه المنطقة الداخلية التي تحتوي على المحتوى.

- المحتوى هو المنطقة الأساسية داخل أي عنصر حيث يظهر محتواه المرئي.
- خصائص `width` و `height` تتحكم بشكل مباشر في أبعاد منطقة المحتوى.
- يتواجد المحتوى داخل الـ `padding` و الـ `border` و الـ `margin`، وهي أجزاء تحيط به من الخارج.

خصائص العرض والتوضع (Display and Positioning) — خصائص العرض (Display): block, inline, inline-block, none

خصائص العرض (Display): block, inline, inline-block, none

تُعد خاصية `display` من أهم خصائص CSS، فهي تحدد كيف يتصرف العنصر في تخطيط الصفحة (Layout). فهمها ضروري لترتيب العناصر بالشكل المطلوب والتحكم في مساحتها وتدفقها.

block

العناصر ذات العرض `block` تشغل عرضها الكامل المتوفر، وتبدأ دائمًا سطرًا جديدًا. لا يمكن لعنصر آخر أن يأتي بجانبها على نفس السطر. يمكن تحديد أبعادها (`width` و `height`) وهوامشها (`margin` و `padding`) من جميع الجهات.

inline

العناصر ذات العرض `inline` تشغل فقط المساحة التي تحتاجها لمحتواها، وتجلس على نفس السطر مع العناصر الأخرى. لا يمكن تحديد أبعاد `width` أو `height` لها مباشرة، كما أن `margin-top` و `margin-bottom` لا تؤثر عليها.

inline-block

العناصر ذات العرض `inline-block` تجمع بين خصائص `inline` و `block`. فهي تجلس على نفس السطر مع العناصر الأخرى مثل `inline`، ولكن يمكن التحكم في أبعادها (`width` و `height`) وهوامشها (`margin` و `padding`) بشكل كامل مثل `block`.

none

عندما تكون قيمة `display` هي `none`، فإن العنصر يختفي تمامًا من الصفحة. لا يشغل أي مساحة، وكأنه غير موجود على الإطلاق في شجرة المستند (DOM)، مما يختلف عن خاصية `visibility: hidden`؛ التي تخفي العنصر ولكن تحتفظ بمساحته.

مثال عملي

```
.block-element {
    display: block;
    background-color: #f8d7da;
    padding: 10px;
    margin-bottom: 5px;
}

.inline-element {
    display: inline;
    background-color: #d1ecf1;
    padding: 5px;
}

.inline-block-element {
    display: inline-block;
    background-color: #d4edda;
    padding: 10px;
    width: 150px;
    margin: 5px;
}

.hidden-element {
    display: none;
}
```

شرح الكود

في هذا المثال، قمنا بتعريف أربعة أنواع من العناصر. العنصر ذو الفئة `block-element` سيشغل سطرًا كاملاً. العنصر ذو الفئة `inline-element` سيتدفق مع النص على نفس السطر. العنصر `inline-block-element` سيظهر على نفس السطر ولكن يمكن تحديد عرضه وارتفاعه. أما العنصر `hidden-element` فلن يظهر في الصفحة على الإطلاق.

- **block**: يشغل سطرًا كاملاً ويسمح بتحديد الأبعاد والهوامش من كل الجهات.
- **inline**: يجلس على نفس السطر مع العناصر الأخرى، ولا يسمح بتحديد العرض أو الارتفاع أو الهامش الرأسي.
- **inline-block**: يجلس على نفس السطر مثل **inline**، لكن يسمح بتحديد الأبعاد والهوامش بالكامل مثل **block**.
- **none**: يخفي العنصر تمامًا من التخطيط ولا يشغل أي مساحة.

خصائص العرض والتوضع (Display and Positioning) — التوضع النسبي (Relative Positioning) والتوضع الثابت (Static Positioning)

التوضع النسبي (Relative Positioning) والتوضع الثابت (Static Positioning) في CSS

في CSS، نتحكم في كيفية وضع العناصر على الصفحة. فهم خصائص التوضع ضروري لتصميم تخطيطات مرنة. سنتعرف هنا على نوعين أساسيين من التوضع: التوضع الثابت والتوضع النسبي.

التوضع الثابت (Static Positioning)

هو التوضع الافتراضي لكل عناصر HTML. العناصر ذات التوضع الثابت تظهر في مكانها الطبيعي ضمن تدفق المستند. خصائص مثل `top`، `bottom`، `left` و `right` لا تؤثر عليها.

التوضع النسبي (Relative Positioning)

يسمح لك بتحريك العنصر من مكانه الطبيعي، لكن **دون التأثير على العناصر الأخرى** حوله. المساحة الأصلية التي كان سيشغلها العنصر تبقى محفوظة. يمكن تحريك العنصر باستخدام `top`، `bottom`، `left` و `right` بالنسبة لموقعه الأصلي.

مثال عملي

```
.box {  
  width: 100px;  
  height: 100px;  
  margin: 10px;  
  border: 1px solid #ccc;
```

```
display: inline-block;
    }

    .static-example {
        position: static;
        background-color: lightblue;
    }

    .relative-example {
        position: relative;
        top: 20px;
        left: 30px;
        background-color: lightgreen;
    }
```

شرح الكود

الكود يوضح صندوقين بحجم موحد. الصندوق ذو الفئة `static-example` يستخدم التوضع الثابت، لذا يظهر في مكانه الطبيعي. أما الصندوق ذو الفئة `relative-example` فيستخدم التوضع النسبي، ويتم تحريكه 20 بكسل إلى الأسفل و30 بكسل إلى اليمين **من موقعه الأصلي**، مع بقاء المساحة الأصلية التي كان سيشغلها شاغرة.

- التوضع **الثابت** هو التوضع الافتراضي ولا يتأثر بإزاحات `top/bottom/left/right`.
- التوضع **النسبي** يحرك العنصر من موقعه الأصلي بناءً على خصائص الإزاحة.
- العنصر ذو التوضع النسبي يترك فراغاً في مكانه الأصلي ضمن تدفق المستند.

خصائص العرض والتوضع (Display and Positioning) — التوضع المطلق (Absolute Positioning) والتوضع اللزج (Fixed/Sticky Positioning)

التوضع المطلق (Absolute Positioning) والتوضع اللزج (Fixed/Sticky Positioning)

تسمح لنا خصائص التوضع في CSS بالتحكم الدقيق في مكان ظهور العناصر على الصفحة، متجاوزين أحياناً التدفق الطبيعي للمحتوى. سنتعلم هنا عن نوعين مهمين من التوضع: المطلق واللزج.

مفهوم التوضع

التوضع المطلق (Absolute Positioning): يخرج العنصر من التدفق الطبيعي للصفحة ويتم توقيعه بالنسبة لأقرب عنصر أب (parent) لديه خاصية توضع (مثل `relative`، `absolute`، `fixed`، أو `sticky`). إذا لم يجد أباً موضحاً، يتم توقيعه بالنسبة للعنصر `<html>`.

التوضع اللزج (Fixed/Sticky Positioning): أيضاً يخرج العنصر من التدفق الطبيعي. يتم توقيعه بالنسبة لنافذة المتصفح (viewport) ويبقى ثابتاً في مكانه حتى عند التمرير (scroll) للصفحة. `sticky` يتصرف كـ `relative` حتى يصل إلى عتبة معينة ثم يصبح `fixed`.

مثال توضيحي (Code Example)

```
.parent-relative {  
  position: relative;  
  width: 300px;  
  height: 150px;  
  border: 2px solid #333;  
  margin-bottom: 200px; /* لضمان ظهور العنصر الثابت عند
```

```

        /* التمرير
        }

        .child-absolute {
            position: absolute;
            top: 20px;
            left: 40px;
            width: 100px;
            height: 50px;
            background-color: #ff6347; /* لون مميز */
            color: white;
            text-align: center;
            line-height: 50px;
        }

        .fixed-element {
            position: fixed;
            bottom: 10px;
            right: 10px;
            width: 120px;
            height: 60px;
            background-color: #4682b4; /* لون مميز */
            color: white;
            text-align: center;
            line-height: 60px;
        }

```

شرح الكود

في هذا المثال، قمنا بتعيين `position: relative` للعنصر الأب (`parent-relative`)، مما يجعله نقطة مرجعية للعنصر المطلق. العنصر `child-absolute` توضع بشكل مطلق داخل الأب، بحيث يبعد 20 بكسل من الأعلى و 40 بكسل من اليسار. أما العنصر `fixed-element`، فقد تم توقيعه بشكل لزج ليبعد 10 بكسل عن أسفل نافذة المتصفح و 10 بكسل عن يمينها، وسيبقى في هذا الموضع حتى مع تمرير الصفحة.

نقاط رئيسية

- `position: absolute`; يضع العنصر بالنسبة لأقرب أب له لديه توضع (positioned ancestor).
- `position: fixed`; يضع العنصر بالنسبة لنافذة المتصفح ويبقى ثابتاً عند التمرير.
- لتحديد موقع العنصر بعد تعيين `position`, نستخدم خصائص مثل `top`, `bottom`, `left`, و `right`.

خصائص العرض والتوضع (Display and Positioning) — التحكم بترتيب العناصر (Z-index)

التحكم بترتيب العناصر (Z-index)

عند تصميم صفحات الويب، قد تتداخل بعض العناصر فوق بعضها البعض، مما يخلق طبقات مرئية. خاصية **z-index** في CSS تمنحك القدرة على تحديد ترتيب هذه العناصر المتداخلة، أي أي عنصر سيظهر في الأعلى (أقرب للمشاهد) وأيها في الأسفل (أبعد).

مفهوم Z-index

تُستخدم خاصية **z-index** لتحديد الترتيب الرأسي (أو عمق التكديس) للعناصر الموضّعة (التي لها خاصية **position** بخلاف **static**). كلما زادت قيمة **z-index**، ظهر العنصر أقرب إلى المشاهد أو فوق العناصر الأخرى ذات القيم الأقل.

مثال عملي

```
.box1 {
  position: relative;
  background-color: lightblue;
  width: 120px;
  height: 120px;
  top: 10px;
  left: 10px;
  z-index: 1;
}

.box2 {
  position: relative;
  background-color: lightcoral;
  width: 120px;
  height: 120px;
  top: 10px;
  left: 10px;
  z-index: 2;
}
```

```
width: 120px;
height: 120px;
top: -60px; /* Overlaps box1 */
left: 60px;
z-index: 2;
}
```

شرح الكود

في هذا المثال، لدينا صندوقان متداخلان: `box1` باللون الأزرق الفاتح و `box2` باللون المرجاني الفاتح. الصندوق `box1` له `z-index: 1`، بينما الصندوق `box2` له `z-index: 2`. بما أن `box2` يمتلك قيمة `z-index` أعلى، فإنه سيظهر فوق `box1`، مما يجعله مرئيًا في المقدمة.

نقاط رئيسية

- تتحكم خاصية `z-index` في ترتيب تراكب العناصر على المحور الرأسي (من الأمام للخلف).
- تعمل هذه الخاصية فقط مع العناصر التي لها خاصية `position` مُحددة (مثل `relative`، `absolute`، `fixed`، `sticky`).
- كلما زادت قيمة `z-index`، ظهر العنصر أعلى (أقرب للمشاهد) من العناصر ذات القيم الأقل.

التصميم المرن (Flexbox) — مقدمة إلى Flexbox ومفهوم المحور الرئيسي والفرعي

مقدمة إلى Flexbox ومفهوم المحور الرئيسي والفرعي

في هذا الدرس، سنتعلم أساسيات Flexbox، وهي أداة قوية في CSS لتنظيم العناصر داخل حاوية واحدة بمرونة وكفاءة. سنستكشف كيف يساعد Flexbox في بناء تصميمات سريعة الاستجابة، ونفهم المفهومين الأساسيين: المحور الرئيسي والمحور الفرعي.

مفهوم Flexbox والمحاور

Flexbox هو نظام تصميم أحادي البعد يسمح لك بترتيب العناصر داخل حاوية (تسمى **Flex Container**) إما في صف واحد أو عمود واحد. يعتمد Flexbox على مفهومين محوريين: **المحور الرئيسي (Main Axis)** وهو الاتجاه الذي تتدفق فيه العناصر، و**المحور الفرعي (Cross Axis)** وهو المحور العمودي على المحور الرئيسي.

مثال عملي

```
.flex-container {
  display: flex;
  flex-direction: row; /* أو column */
  background-color: #f0f0f0;
  padding: 10px;
  border: 1px solid #ccc;
}

.flex-item {
  background-color: #a0c4ff;
  color: white;
  padding: 15px;
  margin: 5px;
```

```
text-align: center;
}
```

شرح الكود

في هذا المثال، قمنا بتحديد حاوية تحمل الفئة `flex-container`. تطبيق الخاصية `display: flex`; يحول هذه الحاوية إلى "حاوية فليكس"، مما يجعل العناصر بداخلها (`flex-item`) عناصر فليكس. تحدد الخاصية `flex-direction: row`; أن المحور الرئيسي سيكون أفقياً (من اليسار إلى اليمين)، وبالتالي ستترتب العناصر جنباً إلى جنب. إذا غيرناها إلى `column`، سيصبح المحور الرئيسي عمودياً وستترتب العناصر فوق بعضها البعض.

- Flexbox هو نظام تصميم أحادي البعد لتنظيم العناصر بمرونة.
- يتم تفعيل Flexbox بتحويل الحاوية إلى `display: flex`;
- يتحكم **المحور الرئيسي** في اتجاه ترتيب العناصر، ويتم تحديده بواسطة الخاصية `flex-direction`.

التصميم المرن (Flexbox) — خصائص الحاوية المرنة (Flex Container): الاتجاه، المحاذاة، التبرير

خصائص الحاوية المرنة (Flex Container): الاتجاه، المحاذاة، التبرير

يُعد نظام Flexbox أداة قوية في CSS لترتيب وتوزيع العناصر داخل حاوية. في هذا الدرس، سنتعلم كيف نتحكم في اتجاه العناصر ومحاذاتها وتبريرها داخل **الحاوية المرنة** باستخدام ثلاث خصائص أساسية.

خصائص الحاوية المرنة

تُعد الحاوية المرنة (Flex Container) هي العنصر الأب الذي يحتوي على العناصر المرنة (Flex Items). هذه الخصائص الثلاث الأساسية تتحكم في كيفية ترتيب وتوزيع العناصر داخل هذه الحاوية على طول المحور الرئيسي (Main Axis) والمحور المتقاطع (Cross Axis):

- **flex-direction**: تُحدد الاتجاه الذي ستتربط وفقه العناصر (صف أفقي أو عمود رأسي).
- **justify-content**: تتحكم في توزيع العناصر على طول المحور الرئيسي.
- **align-items**: تُحدد محاذاة العناصر على طول المحور المتقاطع.

مثال توضيحي

```
.flex-container {  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
  align-items: flex-start;  
}
```

شرح الكود

في هذا المثال، جعلنا العنصر الأب (`flex-container`) حاوية مرنة باستخدام `display: flex`; ثم قمنا بتطبيق الخصائص التالية:

- جعلت العناصر تترتب في صف أفقي. `flex-direction: row`;
- وسّطت العناصر على طول المحور الأفقي (الرئيسي). `justify-content: center`;
- حادت العناصر إلى بداية المحور الرأسي (المتقاطع). `align-items: flex-start`;

نقاط رئيسية

- تتحكم خصائص الحاوية المرنة (`flex-direction`, `justify-content`, `align-items`) في كيفية ترتيب ومحاذاة العناصر بداخلها.
- يُحدد `flex-direction` اتجاه المحور الرئيسي الذي ستترتب عليه العناصر (أفقي أو رأسي).
- يُستخدم `justify-content` للمحاذاة على طول المحور الرئيسي، بينما يُستخدم `align-items` للمحاذاة على طول المحور المتقاطع.

التصميم المرن (Flexbox) — خصائص العناصر المرنة (Flex Items): التمدد، الانكماش، القاعدة

خصائص العناصر المرنة (Flex Items): التمدد، الانكماش، القاعدة

في هذا الدرس، سنتعلم كيف نتحكم بحجم العناصر داخل حاوية Flexbox. سنستكشف الخصائص التي تمنحك القدرة على تحديد كيفية تمدد العنصر، انكماشه، وحجمه الأساسي عند توزيع المساحة.

مفهوم التمدد، الانكماش، والقاعدة (flex-grow, flex-shrink, flex-basis)

تُعد هذه الخصائص أساسية للتحكم في سلوك العناصر المرنة:

- **flex-grow (التمدد):** تحدد مدى قدرة العنصر على التمدد وأخذ مساحة إضافية متوفرة في الحاوية. القيمة الافتراضية هي 0، مما يعني عدم التمدد.
- **flex-shrink (الانكماش):** تحدد مدى قدرة العنصر على الانكماش عند عدم توفر مساحة كافية داخل الحاوية. القيمة الافتراضية هي 1، مما يعني أن العنصر سينكمش عند الحاجة.
- **flex-basis (القاعدة):** تحدد الحجم الأولي للعنصر قبل أي عملية تمدد أو انكماش. يمكن أن تكون قيمة بالبكسل، النسبة المئوية، أو "auto" (تأخذ حجم المحتوى).

يمكن دمج هذه الخصائص الثلاث في خاصية مختصرة واحدة هي **flex** بالترتيب: **flex-grow flex-shrink flex-basis**.

مثال عملي

```
.container {  
  display: flex;  
  width: 400px;
```



```

        height: 100px;
        border: 1px solid #333;
        padding: 5px;
        background-color: #f0f0f0;
    }

    .item {
        background-color: lightblue;
        border: 1px solid blue;
        margin: 5px;
        padding: 10px;
        color: #333;
        font-weight: bold;
        text-align: center;
    }

    .item-1 {
        flex-basis: 100px;
        flex-grow: 1; /* يتمدد ليأخذ جزءًا من المساحة المتبقية */
    }

    .item-2 {
        flex-basis: 200px;
        flex-shrink: 2; /* ينكمش ضعف سرعة الآخرين عند الحاجة */
    }

    .item-3 {
        flex: 1 1 80px; /* انكماش: 1, قاعدة: 80, px يتمدد */
    }

```

شرح الكود

في هذا المثال، لدينا حاوية **container** بثلاثة عناصر **item**. العنصر الأول **item-1** له حجم أساسي 100 بكسل ويسمح له بالتمدد **flex-grow: 1** ليأخذ جزءًا من أي مساحة إضافية. العنصر الثاني **item-2** حجمه الأساسي 200 بكسل، ويسمح له بالانكماش **flex-shrink: 2** بسرعة أكبر من العناصر الأخرى إذا كانت المساحة ضيقة. العنصر الثالث **item-3** يستخدم الخاصية المختصرة **flex: 1 1 80px**، مما يعني أن حجمه الأساسي 80 بكسل، ويمكنه التمدد والانكماش بنسبة 1.

- **flex-grow** تتحكم في كيفية ملء العناصر للمساحة الإضافية.

- **flex-shrink** تتحكم في كيفية تقليص العناصر لحجمها عند نقص المساحة.
- **flex-basis** تحدد الحجم الأولي للعنصر قبل تطبيق قواعد التمدد أو الانكماش.

التصميم المرن (Flexbox) — التفاف العناصر (Wrapping) وتوزيع المساحات

التصميم المرن (Flexbox): التفاف العناصر (Wrapping) وتوزيع المساحات

في هذا الدرس، سنتعلم كيف تتحكم خاصية Flexbox في التفاف العناصر عند عدم وجود مساحة كافية في سطر واحد، وكيفية توزيع المساحات المتبقية بين العناصر.

التفاف العناصر (flex-wrap)

تحدد خاصية `flex-wrap` ما إذا كانت العناصر المرنة يجب أن تلتف إلى سطر جديد عند نفاد المساحة. القيم الشائعة هي `nowrap` (الافتراضي، لا تلتف)، و `wrap` (تلتف إلى سطر جديد)، و `wrap-reverse` (تلتف بترتيب عكسي).

توزيع المساحات (align-content و justify-content)

بعد الالتفاف، يمكننا استخدام `justify-content` لتوزيع العناصر والمساحات على المحور الرئيسي (أفقياً)، و `align-content` لتوزيع المساحات بين صفوف العناصر الملتفة على المحور المتعامد (عمودياً).

مثال توضيحي

```
.flex-container {
  display: flex;
  flex-wrap: wrap; /* السماح للعناصر بالالتفاف */
  justify-content: space-around; /* توزيع العناصر مع مسافات */
  /* حولها */
  align-content: flex-start; /* محاذاة الصفوف الملتفة في */
  /* البداية */
  width: 300px; /* لجعل الالتفاف مرئياً */
}
```

```
border: 2px solid #ccc;
padding: 5px;
height: 200px; /* مرئياً align-content لجعل */
}

.flex-item {
width: 90px;
height: 70px;
background-color: #4CAF50;
margin: 5px;
color: white;
text-align: center;
line-height: 70px;
font-weight: bold;
}
```

شرح الكود

قمنا بإنشاء حاوية مرنة `flex-container`. سمحنا للعناصر بالالتفاف إلى سطر جديد عبر `flex-wrap: wrap`. ثم وزعنا العناصر في السطر باستخدام `justify-content: space-around` لإضافة مسافة متساوية حول كل عنصر. وأخيرًا، استخدمنا `align-content: flex-start` لمحاذاة الصفوف الملتفة في بداية الحاوية.

- `flex-wrap: wrap`; يجعل العناصر تلتف إلى أسطر جديدة عند ضيق المساحة.
- `justify-content` يتحكم في توزيع العناصر والمساحات على المحور الرئيسي (أفقياً).
- `align-content` يتحكم في توزيع المساحات بين صفوف العناصر الملتفة على المحور المتعامد (عمودياً).

شبكة (CSS Grid) CSS — مقدمة إلى CSS Grid

وتعريف الشبكة

مقدمة إلى CSS Grid وتعريف الشبكة

تُعد شبكة CSS Grid نظام تخطيط قويًا يُمكنك من تصميم صفحات الويب بشكل ثنائي الأبعاد، مُتحكمًا في وضع العناصر عبر صفوف وأعمدة. تُمكنك هذه الأداة من إنشاء تخطيطات معقدة ومرنة بسهولة.

تفعيل وتعريف الشبكة

لتحويل أي عنصر في صفحتك إلى حاوية شبكة (Grid container)، يجب استخدام الخاصية `display: grid`. بعد تفعيل الشبكة، يمكنك تحديد هيكلها بوضوح عن طريق تعريف عدد الأعمدة والصفوف وأحجامها.

مثال على تعريف الشبكة

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: auto 150px;
  gap: 15px;
}
```

شرح الكود

هذا الكود يحول العنصر الذي يحمل الفئة `container` إلى حاوية شبكة. تُفعّل الخاصية `display: grid`; نظام الشبكة. تحدد `grid-template-columns: 1fr 1fr 1fr`; ثلاث أعمدة متساوية في الحجم، بينما تحدد `grid-template-rows: auto 150px`; صفين؛ الأول بارتفاع تلقائي والثاني بارتفاع ثابت 150 بكسل. تضيف `gap: 15px`; مسافة 15 بكسل بين خلايا الشبكة.

- **CSS Grid** يمنحك تحكمًا كاملاً في تصميم التخطيطات ثنائية الأبعاد (الصفوف والأعمدة).
- لتفعيل الشبكة على عنصر، استخدم الخاصية `display: grid`.
- حدد هيكل الشبكة (الأعمدة والصفوف) باستخدام `grid-template-columns` و `grid-template-rows`.

شبكة (CSS Grid) CSS — تحديد الأعمدة والصفوف (grid-template-columns, grid-template-rows)

تحديد الأعمدة والصفوف (grid-template-columns, grid-template-rows)

في شبكة (CSS Grid) CSS، يمكنك تنظيم المحتوى بسهولة في صفوف وأعمدة. هذا الدرس يشرح كيف تحدد بنية هذه الأعمدة والصفوف باستخدام خاصيتين أساسيتين.

المفهوم الأساسي

تُستخدم الخاصية **grid-template-columns** لتحديد عدد الأعمدة وعرض كل منها. بينما تُستخدم الخاصية **grid-template-rows** لتحديد عدد الصفوف وارتفاع كل منها. يمكنك استخدام وحدات مختلفة مثل **fr**، **%**، **px**، و **auto** لتحديد الأحجام.

مثال توضيحي

```
.container {  
  display: grid;  
  grid-template-columns: 100px 1fr 100px;  
  grid-template-rows: 50px auto;  
}
```

شرح الكود

في هذا المثال، جعلنا العنصر **container** حاوية شبكة (Grid Container) باستخدام **display: grid**؛ ثم:

- `grid-template-columns: 100px 1fr 100px`; هذا ينشئ ثلاثة أعمدة. العمود الأول عرضه 100 بكسل، والثاني يأخذ المساحة المتبقية بالتساوي (1fr تعني "وحدة كسرية واحدة")، والثالث عرضه 100 بكسل.
- `grid-template-rows: 50px auto`; هذا ينشئ صفين. الصف الأول ارتفاعه 50 بكسل، والصف الثاني يتكيف تلقائيًا ليحتوي المحتوى (auto).

خلاصة الدرس

- تُحدد بنية الأعمدة في شبكة CSS باستخدام الخاصية `grid-template-columns`.
- تُحدد بنية الصفوف في شبكة CSS باستخدام الخاصية `grid-template-rows`.
- يمكنك استخدام وحدات قياس مرنة (مثل `fr` و `%` و `auto`) أو ثابتة (مثل `px`) لتحديد أحجام الأعمدة والصفوف بدقة.

شبكة CSS (CSS Grid) — وضع العناصر داخل الشبكة (grid-column, grid-row)

وضع العناصر داخل الشبكة (grid-column, grid-row)

في درس شبكة CSS (CSS Grid)، نتعلم كيف نضع العناصر داخل الشبكة بشكل دقيق ومنظم. تُساعدنا الخاصيتان **grid-column** و **grid-row** في تحديد المكان الذي سيشغله العنصر من أعمدة و صفوف الشبكة بالضبط.

مفهوم رئيسي

grid-column: تُستخدم لتحديد العمود أو الأعمدة التي سيبدأ العنصر وينتهي عندها داخل الشبكة.
grid-row: تُستخدم لتحديد الصف أو الصفوف التي سيبدأ العنصر وينتهي عندها داخل الشبكة.
يمكن تحديد هذه المواقع باستخدام أرقام خطوط الشبكة (grid lines) أو باستخدام كلمة **span** لتحديد عدد الأعمدة/الصفوف التي يمتد بها العنصر.

مثال عملي

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* شبكة بـ 3 أعمدة
  * متساوية */
  grid-template-rows: repeat(2, 100px); /* شبكة بـ 2 صفوف
  * بارتفاع 100px */
  gap: 10px;
}

.item-a {
  background-color: lightblue;
  grid-column: 1 / 3; /* يبدأ من خط العمود 1 وينتهي قبل خط
  * العمود 3 */
  grid-row: 1 / span 1; /* يبدأ من خط الصف 1 ويمتد لصف واحد */
}
```

```

    }

    .item-b {
        background-color: lightcoral;
        grid-column: 3 / span 1; /* ويمتد لعمود
        واحد */
        grid-row: 1 / 3; /* وينتهي قبل خط
        الصف 3 */
    }

```

شرح الكود

في هذا المثال، أنشأنا حاوية شبكية بها 3 أعمدة و 2 صفوف.

للعنصر **item-a**: استخدمنا `grid-column: 1 / 3`; لجعله يشغل العمودين الأول والثاني (من الخط الأول وحتى قبل الخط الثالث). واحتل `grid-row: 1 / span 1`; الصف الأول فقط.

للعنصر **item-b**: استخدمنا `grid-column: 3 / span 1`; لجعله يشغل العمود الثالث فقط (يبدأ من الخط الثالث ويمتد لعمود واحد). واحتل `grid-row: 1 / 3`; الصفين الأول والثاني (من الخط الأول وحتى قبل الخط الثالث).

- تُستخدم **grid-column** للتحكم في موضع العنصر وعرضه عبر أعمدة الشبكة.
- تُستخدم **grid-row** للتحكم في موضع العنصر وارتفاعه عبر صفوف الشبكة.
- يمكن تحديد النطاق باستخدام أرقام خطوط البداية والنهاية، أو باستخدام **span** لتحديد عدد المسارات التي يمتدها العنصر.

شبكة (CSS Grid) — مناطق الشبكة (Grid) (Areas) والفجوات (Gaps)

مناطق الشبكة (Grid Areas) والفجوات (Gaps)

تُعد مناطق الشبكة والفجوات أدوات قوية في شبكة (CSS Grid) CSS تمنحك تحكمًا كبيرًا في تصميم التخطيط. تسمح لك **مناطق الشبكة** بتسمية أجزاء من شبكتك وجعل العناصر تشغل تلك المناطق بسهولة، بينما تُستخدم **الفجوات** لإضافة مسافات بين صفوف وأعمدة الشبكة.

مناطق الشبكة (Grid Areas)

باستخدام خاصية `grid-template-areas` على الحاوية، يمكنك تحديد تخطيط مرئي للشبكة بتسمية كل خلية. بعد ذلك، يمكنك تعيين اسم المنطقة لكل عنصر فرعي باستخدام خاصية `grid-area` لجعل العنصر يشغل تلك المنطقة المحددة.

الفجوات (Gaps)

تُستخدم الفجوات لإضافة مسافات بيضاء بين خلايا الشبكة. يمكنك استخدام `grid-gap` لتعيين فجوة متساوية بين الصفوف والأعمدة، أو استخدام `row-gap` و `column-gap` لضبط المسافات بشكل منفصل.

مثال عملي

```
.grid-container {
  display: grid;
  grid-template-columns: 1fr 2fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-gap: 15px;
```

```

    }

    .header {
      grid-area: header;
      background-color: #f8d7da;
    }

    .sidebar {
      grid-area: sidebar;
      background-color: #d1ecf1;
    }

    .main {
      grid-area: main;
      background-color: #d4edda;
    }

    .footer {
      grid-area: footer;
      background-color: #f8d7da;
    }

```

شرح الكود

في هذا المثال، قمنا أولاً بتعريف حاوية شبكة ذات عمودين وثلاثة صفوف. استخدمنا `grid-template-areas` لإنشاء تخطيط مرئي، حيث يشغل **"header"** الصف الأول بالكامل، و **"sidebar"** و **"main"** يشغلان الصف الثاني، و **"footer"** يشغل الصف الثالث بالكامل. ثم عيّنا لكل عنصر (header, sidebar, main, footer) منطقة الشبكة الخاصة به باستخدام `grid-area`. أخيراً، طبقنا `grid-gap: 15px`; لإضافة مسافة 15 بكسل بين جميع صفوف وأعمدة الشبكة، مما يوفر تباعدًا بصريًا جيدًا.

- **مناطق الشبكة (Grid Areas):** تمنحك القدرة على تسمية أجزاء من تخطيطك وتعيين العناصر لها بسهولة.
- **grid-template-areas:** تُستخدم على الحاوية لتحديد التخطيط المرئي بأسماء المناطق.
- **الفجوات (Gaps):** تُستخدم لإضافة مسافات بيضاء بين عناصر الشبكة لزيادة قابلية القراءة والجمالية.

التصميم المتجاوب (Responsive Design) —

مقدمة للتصميم المتجاوب وأهميته

html

مقدمة للتصميم المتجاوب وأهميته

في هذا الدرس، سنتعرف على مفهوم التصميم المتجاوب (Responsive Design) في CSS، وهو أساس بناء مواقع ويب تعمل بفعالية وتظهر بشكل جذاب على جميع الأجهزة والشاشات المختلفة، من الحواسيب المكتبية الكبيرة إلى الهواتف الذكية الصغيرة، مما يضمن تجربة مستخدم ممتازة.

ما هو التصميم المتجاوب؟

التصميم المتجاوب هو منهجية في تصميم وتطوير الويب تجعل محتوى الصفحة يتكيف تلقائياً ويُعاد ترتيبه ليناسب حجم ونوع الشاشة التي تُعرض عليها. الهدف الأساسي هو تقديم أفضل تجربة عرض وتفاعل للمستخدمين، بغض النظر عن الجهاز الذي يستخدمونه.

مثال عملي (استعلامات الوسائط - Media Queries)

```
@media screen and (max-width: 768px) {  
    body {  
        background-color: #f0f8ff; /* لون أزرق فاتح */  
    }  
    h1 {  
        font-size: 2em;  
        text-align: center;  
    }  
}
```

شرح الكود

يستخدم هذا الكود [استعلام الوسائط \(media@\)](#) لتطبيق أنماط CSS محددة فقط عندما يكون عرض الشاشة 768 بكسل أو أقل (مثل الأجهزة اللوحية والهواتف). في هذه الحالة، يتم تغيير لون خلفية الصفحة إلى الأزرق الفاتح، ويصبح حجم خط العنوان 2em مع محاذاة في المنتصف، مما يحسن من قراءة وتجربة المحتوى على الشاشات الأصغر حجماً.

نقاط رئيسية

- التصميم المتجاوب ضروري لضمان ظهور موقعك بشكل صحيح وجذاب على جميع أنواع الأجهزة.
- يعمل على تحسين تجربة المستخدم (UX) بشكل كبير ويجعل تصفح الموقع سهلاً وممتعاً.
- تعتبر [استعلامات الوسائط \(Media Queries\)](#) هي الأداة الأساسية في CSS لتحقيق التصميم المتجاوب.

...

التصميم المتجاوب (Responsive Design) — استعلامات الوسائط (Media Queries)

استعلامات الوسائط (Media Queries)

تُعد استعلامات الوسائط (Media Queries) حجر الزاوية في التصميم المتجاوب (Responsive Design) باستخدام CSS. هي تسمح لموقعك بالظهور بشكل رائع والتكيف مع أحجام الشاشات المختلفة، من الهواتف الذكية إلى أجهزة الكمبيوتر المكتبية.

ما هي استعلامات الوسائط؟

هي قواعد CSS خاصة تسمح لك بتطبيق أنماط معينة فقط عندما تتطابق خصائص معينة للجهاز الذي يتصفح الموقع، مثل عرض الشاشة أو ارتفاعها أو نوع الجهاز. بفضلها، يمكنك تصميم تجربة مستخدم مثالية لكل حجم شاشة.

مثال على الكود

```
/* Default styles */
body {
  background-color: lightblue;
  font-size: 16px;
}

/* Styles for screens smaller than 600px */
@media screen and (max-width: 600px) {
  body {
    background-color: lightcoral;
    font-size: 14px;
  }
}
```

شرح الكود

يحدد الجزء الأول من الكود الأنماط الافتراضية للجسم (**body**)، وهي خلفية زرقاء فاتحة وحجم خط 16 بكسل. بعد ذلك، يأتي استعلام الوسائط `@media screen and (max-width: 600px)` الذي يخبر المتصفح بتطبيق الأنماط الداخلية فقط إذا كان الجهاز شاشة وعرضه الأقصى 600 بكسل أو أقل. في هذه الحالة، تتغير خلفية الجسم إلى الأحمر المرجاني وحجم الخط إلى 14 بكسل، مما يظهر كيف تتغير الأنماط للشاشات الصغيرة.

- **أساس التصميم المتجاوب:** استعلامات الوسائط هي الأداة الرئيسية لجعل تصميم موقعك يتكيف مع أحجام الشاشات المختلفة.
- **تحديد الشروط:** تسمح لك بتطبيق أنماط CSS محددة فقط عندما تتحقق شروط معينة (مثل عرض الشاشة).
- **تحسين تجربة المستخدم:** تساعد في تقديم تجربة ممتازة للمستخدم بغض النظر عن الجهاز الذي يستخدمه.

التصميم المتجاوب (Responsive Design) —

وحدات القياس المرنة: em, rem, vw, vh

وحدات القياس المرنة في التصميم المتجاوب

في عالم التصميم المتجاوب (Responsive Design)، نحتاج أن نتكيف عناصر صفحة الويب مع أحجام الشاشات المختلفة. نتعلم هنا وحدات القياس المرنة في CSS مثل **em**، **rem**، **vw** و **vh** التي تجعل تصميمك يتجاوب بذكاء.

لماذا نستخدم وحدات القياس المرنة؟

على عكس وحدات القياس الثابتة مثل **px**، تتغير الوحدات المرنة تلقائياً بناءً على عامل معين (مثل حجم خط العنصر الأب، حجم خط العنصر الجذري، أو أبعاد شاشة العرض). هذا يضمن ظهور تصميمك بشكل ممتاز على أي جهاز.

وحدات القياس المرنة

- **em**: تعتمد على حجم الخط للعنصر **الأب** المباشر. إذا لم يحدد حجم خط للأب، يرثه من العنصر الأب الأعلى، وهكذا.
- **rem**: تعتمد على حجم الخط للعنصر **الجذري** (**html**). هذا يجعل التحكم في حجم الخط أسهل وأكثر اتساقاً.
- **vw**: (Viewport Width) تمثل 1% من **عرض** نافذة المتصفح. على سبيل المثال، **50vw** تعني 50% من عرض الشاشة.
- **vh**: (Viewport Height) تمثل 1% من **ارتفاع** نافذة المتصفح. على سبيل المثال، **50vh** تعني 50% من ارتفاع الشاشة.

مثال الكود

```
html {
```

```
font-size: 16px; /* حجم الخط الأساسي للعنصر الجذري */
}

.heading {
  font-size: 2rem; /* ضعف حجم خط العنصر الجذري 2 (32px) */
  margin-bottom: 0.5em; /* ضعف حجم خط 0.5 (16px) */
}

.text-content {
  font-size: 1em; /* نفس حجم خط العنصر الأب (إذا كان) */
  /* (داخل عنصر بحجم خط معين content) */
  width: 90vw; /* من عرض الشاشة 90% */
  height: 30vh; /* من ارتفاع الشاشة 30% */
  padding: 1rem;
  background-color: #f5f5f5;
}
```

شرح الكود

يبدأ الكود بتحديد حجم الخط الأساسي لعنصر `html` ليصبح 16 بكسل، وهو ما تستند إليه وحدة `rem`. عنوان `heading` سيأخذ حجم خط `2rem` (أي ضعف حجم خط `html`) بينما الهامش السفلي `0.5em` سيتناسب مع حجم خط العنوان نفسه. قسم المحتوى `text-content` سيأخذ حجم خط `1em` (الذي سيرثه من أبيه)، ويحدد عرضه بنسبة 90% من عرض الشاشة وارتفاعه بنسبة 30% من ارتفاعها باستخدام `vw` و `vh`، مما يجعله يتكيف مع حجم الشاشة.

- **المرونة هي المفتاح:** استخدام هذه الوحدات يجعل تصميمك يتأقلم تلقائيًا مع أي حجم شاشة.
- **rem للتحكم العام:** مثالية لتحديد أحجام الخطوط الأساسية في المشروع لأنها تعتمد على عنصر `html`.
- **vw و vh للأبعاد الكبيرة:** ممتازة لتحديد أبعاد الكتل والعناصر الكبيرة التي يجب أن تتناسب مع أبعاد الشاشة.

التصميم المتجاوب (Responsive Design) — الصور المتجاوبة (Responsive Images)

الصور المتجاوبة (Responsive Images)

الصور هي جزء أساسي من أي موقع ويب. لضمان تجربة مستخدم ممتازة على جميع الأجهزة، يجب أن تتجاوب الصور مع أحجام الشاشات المختلفة. هذا الدرس سيوضح كيف نجعل الصور متجاوبة باستخدام CSS، مما يضمن عرضها بشكل صحيح وجذاب بغض النظر عن الجهاز المستخدم.

لماذا الصور المتجاوبة مهمة؟

الصور المتجاوبة هي الصور التي تتكيف تلقائيًا مع حجم الشاشة ودقتها. هذا يمنع الصور من الظهور كبيرة جدًا أو صغيرة جدًا، ويحافظ على تصميم الموقع سليمًا، ويحسن أداء التحميل وتجربة المستخدم بشكل عام من خلال تجنب تحميل صور بحجم غير ضروري.

مثال على الكود

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

شرح الكود

هذا الكود يطبق قاعدتين أساسيتين على جميع الصور:

- `max-width: 100%` تضمن أن الصورة لن تتجاوز عرض العنصر الحاوي لها أبدًا، وفي نفس الوقت ستمتد لتتلاءم 100% من عرضه إذا كان العنصر الحاوي أصغر من عرض الصورة الأصلي.
- `height: auto` تحافظ على نسبة أبعاد الصورة الأصلية، فلا تتشوه عند تغيير حجمها.

النقاط الأساسية

- الصور المتجاوبة تتكيف تلقائيًا مع أحجام الشاشات المختلفة.
- تمنع قاعدة `max-width: 100%` الصور من الخروج عن حدود التصميم أو التسبب في أشرطة تمرير أفقية.
- `height: auto` تحافظ على جودة الصورة ونسبة أبعادها الأصلية أثناء التغيير.

المحددات المتقدمة والفئات/العناصر الزائفة

— المحددات الزائفة للفئة (Pseudo-classes):

hover, focus, nth-child

المحددات الزائفة للفئة (Pseudo-classes): hover, focus, nth-child

المحددات الزائفة للفئة في CSS تسمح لنا بتنسيق العناصر بناءً على حالتها الخاصة أو موقعها في شجرة المستند، بدلاً من مجرد اسمها أو معرفها أو فئتها. هذا يضيف ديناميكية ومرونة كبيرة لتصميم واجهات المستخدم.

المحددات الزائفة للفئة (Pseudo-classes)

هي كلمات مفتاحية تبدأ بنقطتين رأسيين (:). وتُضاف إلى المحددات العادية لاستهداف عنصر عندما يكون في حالة معينة. سنتعلم عن ثلاث منها:

- **hover:** لتنسيق العنصر عندما يمر مؤشر الفأرة فوقه.
- **focus:** لتنسيق العنصر عندما يكون هو العنصر النشط أو المحدد، مثل حقل إدخال.
- **nth-child(n):** لاختيار العناصر بناءً على ترتيبها بين إخوتها.

مثال الكود

```
.button-like {
  background-color: #007bff;
  color: white;
  padding: 10px 15px;
  border: none;
  cursor: pointer;
  transition: background-color 0.3s ease;
}
```

```

        .button-like:hover {
            background-color: #0056b3;
        }

        input[type="text"] {
            border: 1px solid #ccc;
            padding: 8px;
            transition: border-color 0.3s ease, box-shadow 0.3s ease;
        }

        input[type="text"]:focus {
            border-color: #007bff;
            box-shadow: 0 0 5px rgba(0, 123, 255, 0.5);
            outline: none;
        }

        ul li {
            padding: 5px 0;
        }

        ul li:nth-child(odd) {
            background-color: #f0f8ff; /* Light blue */
        }

        ul li:nth-child(even) {
            background-color: #f8f8f8; /* Light gray */
        }

```

شرح الكود

- **button-like:hover** يغير لون خلفية العنصر ذي الفئة **button-like** عندما يمر مؤشر الفأرة فوقه، مما يعطي تأثيرًا بصريًا تفاعليًا.
- **input[type="text"]:focus** يغير حدود حقل الإدخال ولون الظل المحيط به عند النقر عليه أو تحديده ليصبح نشطًا.
- **ul li:nth-child(odd)** و **ul li:nth-child(even)** يحددان خلفيات مختلفة للعناصر الفردية والزوجية ضمن قائمة **ul**، مما يسهل قراءة القوائم الطويلة.

ملخص الفكرة

- المحددات الزائفة للفئة تسمح بتغيير مظهر العناصر بناءً على حالتها التفاعلية أو موقعها.
- **hover:** يستخدم لتأثيرات عند مرور الفأرة، و **focus:** للعناصر النشطة (مثل حقول الإدخال).
- **nth-child: ()** يوفر طريقة قوية لتنسيق العناصر بترتيب معين ضمن مجموعة أخواتها.

المحددات المتقدمة والفئات/العناصر الزائفة

— المحددات الزائفة للعنصر (Pseudo-)

before, after, first-line: (elements)

المحددات الزائفة للعنصر (Pseudo-elements): قبل،

بعد، السطر الأول

في CSS، تسمح لنا المحددات الزائفة بتنسيق أجزاء معينة من عنصر لا تُمثّل بواسطة وسم HTML صريح. إنها طريقة قوية لإضافة تأثيرات جمالية أو محتوى إضافي دون الحاجة لتعديل بنية HTML الأساسية.

مفهوم المحددات الزائفة: before, ::after, ::first-line:

تُستخدم هذه المحددات لتعديل أجزاء افتراضية من العنصر:

- **before::**: لإضافة محتوى افتراضي قبل المحتوى الفعلي للعنصر.
- **after::**: لإضافة محتوى افتراضي بعد المحتوى الفعلي للعنصر.
- **first-line::**: لتنسيق السطر الأول فقط من النص داخل عنصر كتلوي.

مثال عملي

```
p::before {
  content: "مقدمة: ";
  color: blue;
  font-weight: bold;
}

p::after {
  content: ".انتهى - ";
  color: green;
}
```



```
}  
  
p::first-line {  
    color: red;  
    font-size: 1.2em;  
}
```

شرح الكود

في هذا المثال، قمنا بتطبيق:

- `p::before`: يضيف كلمة "مقدمة:" باللون الأزرق الغامق قبل نص كل فقرة.
- `p::after`: يضيف نص " - انتهى." باللون الأخضر بعد نهاية نص كل فقرة.
- `p::first-line`: يجعل السطر الأول من نص الفقرة باللون الأحمر وبحجم خط أكبر. يُطبّق هذا التنسيق على السطر الأول الذي يظهر فعليًا في المتصفح، والذي قد يتغير حسب عرض الشاشة.

نقاط رئيسية

- المحددات الزائفة تسمح لك بتنسيق أجزاء من عنصر لا توجد كعناصر HTML مستقلة.
- `before::` و `after::` يتطلبان خاصية `content` لإضافة محتوى.
- `first-line::` ينسق فقط السطر الأول المعروض من النص داخل العنصر الكتلوي.

المحددات المتقدمة والفئات/العناصر الزائفة

— محددات العلاقة (Combinators المتقدمة)

محددات العلاقة (Combinators المتقدمة)

في CSS، يمكننا محددات العلاقة من استهداف العناصر ليس فقط باسمها أو معرفها، بل بناءً على علاقتها بعناصر أخرى في بنية HTML. هذا يمنحنا تحكمًا دقيقًا وقويًا في تصميم صفحات الويب.

محددات العلاقة المتقدمة

محددات العلاقة المتقدمة تساعد في تحديد العناصر الشقيقة بناءً على موقعها. سنتعلم عن المحدد الشقيق المجاور (+) والمحدد الشقيق العام (~).

- **المحدد الشقيق المجاور (+):** يختار أول عنصر شقيق يأتي مباشرة بعد عنصر آخر. يجب أن يكونا في نفس المستوى ولهما نفس الأب.
- **المحدد الشقيق العام (~):** يختار جميع العناصر الشقيقة التي تتبع عنصرًا معينًا في نفس المستوى، بغض النظر عن وجود عناصر أخرى بينهما.

مثال عملي

```
--> مشابه لهذا HTML افترض هيكـ <!-->
<div>
  <p>هذه فقرة</p>
  <span>هذا عنصر سبان مجاور</span>
  <p>فقرة أخرى</p>
  <span>هذا عنصر سبان آخر</span>
  <span>وهذا سبان ثالث</span>
</div>

/* كود CSS */
p + span {
  background-color: yellow;
  border: 1px solid orange;
```

```
}  
  
p ~ span {  
  color: blue;  
  font-style: italic;  
}
```

شرح الكود

في المثال أعلاه، المحدد `p + span` سيطبق خلفية صفراء وحدود برتقالية فقط على عنصر `<span>` الأول الذي يتبع مباشرة عنصر `<p>` (وهو "هذا عنصر سبان مجاور").

أما المحدد `p ~ span` فسيطبق لونًا أزرق وخطًا مائلًا على جميع عناصر `<span>` التي تتبع أي عنصر `<p>` في نفس المستوى، بما في ذلك العنصر المجاور (مما قد يغير لونه من الأحمر إلى الأزرق إذا طبق بعده) و"هذا عنصر سبان آخر." و"وهذا سبان ثالث."

ملخص الفكرة

- تستخدم محددات العلاقة لاستهداف العناصر بناءً على علاقتها الهيكلية بعناصر أخرى.
- المحدد الشقيق المجاور (+) يختار العنصر الشقيق المباشر التالي فقط.
- المحدد الشقيق العام (~) يختار جميع العناصر الشقيقة التالية لنفس العنصر، بغض النظر عن الترتيب.

المحددات المتقدمة والفئات/العناصر الزائفة

— محددات النفي والتطابق (:is, :not())

محددات النفي والتطابق (:is, :not())

تُعد محددات النفي والتطابق من الأدوات القوية في CSS التي تمنحك مرونة أكبر في اختيار العناصر وتطبيق التنسيقات. إنها تساعد على كتابة كود CSS أنظف وأكثر فعالية، خاصة عند التعامل مع مجموعات كبيرة من العناصر أو استثناء بعضها.

محدد النفي (:not())

يسمح لك هذا المحدد باختيار جميع العناصر التي **لا** تتطابق مع المحدد الذي تعطيه بداخله. إنه مفيد جدًا لتطبيق تنسيق على مجموعة من العناصر مع استثناء بعضها بشكل محدد.

محدد التطابق (:is)

يسمح لك هذا المحدد باختيار العناصر التي تتطابق مع **أي واحد** من قائمة المحددات التي تعطيه بداخله. يجعل الكود أقصر وأسهل في القراءة عندما تريد تطبيق نفس التنسيق على عدة محددات مختلفة.

مثال عملي

```
p:not(.highlight) {  
    color: gray;  
}  
  
:is(h1, h2, h3) {  
    color: #2c3e50;  
    font-family: 'Amiri', serif;  
}
```

شرح الكود

في هذا المثال، قمنا بتحديد لون النص ليكون رماديًا `gray` لكل الفقرات `<p>` **باستثناء** تلك التي تحمل الفئة `highlight`. أما الجزء الثاني، فيقوم بتعيين لون الخط `#2c3e50` ونوع الخط `'Amiri', serif` لكل من عناوين `<h1>` و `<h2>` و `<h3>` في سطر واحد باستخدام `is:()`.

- `not:()` تزيد من دقة التحكم، مما يسمح باستثناء عناصر معينة من التنسيق العام.
- `is:()` تجعل كود CSS أقصر وأكثر قابلية للقراءة عند استهداف عدة محددات بنفس التنسيق.
- كلتا الأدوات تعززان من قدرة المطور على كتابة محددات CSS مرنة وفعالة.

الانتقالات والتحويلات (Transitions and Transforms) — الانتقالات في CSS (Transitions): الخاصية، المدة، دالة التوقيت، التأخير

الانتقالات في CSS (Transitions)

تُمكنك انتقالات CSS من إنشاء تأثيرات حركية سلسلة عند تغيير خصائص العناصر. بدلاً من التغيير المفاجئ، تنتقل القيم تدريجياً بمرور الوقت، مما يُحسن تجربة المستخدم ويجعل واجهة الموقع أكثر جاذبية.

خصائص الانتقال الأساسية

لإنشاء انتقال، تحتاج إلى تحديد أربعة خصائص رئيسية في CSS:

- **transition-property**: تحدد الخاصية التي ستتأثر بالانتقال (مثل `background-color`، `width`).
- **transition-duration**: تحدد مدة الانتقال بالثواني (s) أو المللي ثانية (ms).
- **transition-timing-function**: تحدد كيفية تقدم سرعة الانتقال بمرور الوقت (مثل `ease`، `linear`، `ease-in-out`).
- **transition-delay**: تحدد فترة الانتظار قبل بدء الانتقال، أيضاً بالثواني أو المللي ثانية.

مثال توضيحي

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: blue;  
  transition-property: background-color, width;
```

```
        transition-duration: 0.7s, 0.4s;
        transition-timing-function: ease-out;
        transition-delay: 0s, 0.1s;
    }

    .box:hover {
        background-color: red;
        width: 200px;
    }
```

شرح الكود

لدينا عنصر `box` بعرض ولون خلفية معينين. عند التحويم (`hover:`) فوقه، يتغير لون الخلفية إلى الأحمر وعرضه إلى 200px.

تُحدد `transition-property` أن الانتقال سيحدث على خصائص `background-color` و `width`. تُعيّن `transition-duration` مدة انتقال اللون بـ 0.7 ثانية، والعرض بـ 0.4 ثانية. تُحدد `transition-timing-function` أن الانتقال سيكون سلساً ويبدأ سريعاً ثم يتباطأ. أما `transition-delay`، فتُجعل انتقال اللون يبدأ فوراً (0s)، بينما يبدأ انتقال العرض بعد 0.1 ثانية من التحويم.

- تُحول الانتقالات التغييرات المفاجئة في خصائص CSS إلى حركات مرئية وسلسة.
- تتحكم الخصائص الأربعة (`property`, `duration`, `timing-function`, `delay`) في كل جانب من جوانب الانتقال.
- يمكن تطبيق الانتقالات على خاصية واحدة أو عدة خصائص في نفس الوقت.

الانتقالات والتحويلات (Transitions and Transforms) — التحويلات ثنائية الأبعاد (2D Transforms): translate, rotate, scale, skew

التحويلات ثنائية الأبعاد (2D Transforms) في CSS

تُمكنك التحويلات ثنائية الأبعاد في CSS من تغيير موضع وحجم وزاوية وميل العناصر في مستوى ثنائي الأبعاد (X و Y). هذه التحويلات أدوات قوية لجعل صفحات الويب أكثر تفاعلية وجاذبية بصرية دون التأثير على تدفق المستند.

فهم التحويلات ثنائية الأبعاد

الخاصية **transform** في CSS هي مفتاح تطبيق هذه التحويلات. سنتعرف على أربع وظائف رئيسية:

- **translate()**: لتحريك العنصر من مكانه.
- **rotate()**: لتدوير العنصر حول نقطة محورية.
- **scale()**: لتغيير حجم العنصر (تكبيره أو تصغيره).
- **skew()**: لإمالة العنصر بزاوية معينة.

مثال عملي

```
.box {
  width: 100px;
  height: 100px;
  background-color: #3498db;
  margin: 50px;
  transition: transform 0.3s ease-in-out;
}
```



```
.box:hover {  
  transform: translateX(30px) rotate(45deg) scale(1.2)  
            skewY(10deg);  
}
```

شرح الكود

في هذا المثال، لدينا صندوق أزرق بسيط. عند مرور مؤشر الفأرة فوق هذا الصندوق (`hover:`)، سيحدث التالي:

- `translateX(30px)`: سيتحرك الصندوق 30 بكسل إلى اليمين.
- `rotate(45deg)`: سيدور الصندوق 45 درجة في اتجاه عقارب الساعة.
- `scale(1.2)`: سيزداد حجم الصندوق بنسبة 20% (يصبح 1.2 ضعف حجمه الأصلي).
- `skewY(10deg)`: سيميل الصندوق 10 درجات على المحور Y.

الخاصية `transition` تجعل هذه التحويلات تحدث بسلسلة على مدار 0.3 ثانية.

ملخص مفيد

- تُستخدم خاصية `transform` لتغيير موضع، حجم، تدوير، أو إمالة العناصر في بعدين.
- الوظائف الأساسية هي `scale()`، `rotate()`، `translate()`، و `skew()`.
- التحويلات ثنائية الأبعاد تزيد من جاذبية وتفاعل واجهة المستخدم دون التأثير على تدفق باقي المحتوى.

الانتقالات والتحويلات (Transitions and Transforms)

التحويلات ثلاثية الأبعاد (3D Transforms)

rotateX, rotateY, perspective : (Transforms)

التحويلات ثلاثية الأبعاد (3D Transforms)

تضيف التحويلات ثلاثية الأبعاد (3D Transforms) عمقًا وحياة لعناصر الويب، مما يسمح لك بتحريك العناصر في محاور X و Y و Z. تخلق هذه التحويلات تأثيرات بصرية جذابة وكأن العنصر يخرج من الشاشة أو يدور في الفراغ.

المفاهيم الأساسية

rotateX(): يدور العنصر حول المحور الأفقي (X-axis).

rotateY(): يدور العنصر حول المحور العمودي (Y-axis).

perspective(): تحدد عمق المنظور ثلاثي الأبعاد، مما يعطي إحساسًا بالعمق للأجسام المتحولة. كلما كانت قيمة البكسل أقل (أقرب إلى الصفر)، زاد تأثير العمق.

مثال عملي

```
.box {
  width: 120px;
  height: 120px;
  background-color: #6a5acd; /* بنفسجي فاتح */
  margin: 50px auto;
  transition: transform 0.6s ease;
  border-radius: 8px;
}

.box:hover {
  transform: perspective(600px) rotateX(60deg) rotateY(30deg);
}
```

```
background-color: #483d8b; /* بنفسجي غامق */
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء مربع. عند تمرير مؤشر الفأرة فوقه (`box: hover`), يتم تطبيق التحويلات:

- `transform: perspective(600px)`: تحدد عمق المنظور بمقدار 600 بكسل، مما يجعل العنصر يبدو وكأنه يتراجع في الفضاء.
- `rotateX(60deg)`: يدور المربع 60 درجة حول المحور الأفقي.
- `rotateY(30deg)`: يدور المربع 30 درجة حول المحور العمودي.
- `transition: transform 0.6s ease`: يجعل التحويلات سلسلة وتستغرق 0.6 ثانية.

نقاط مهمة

- `rotateY` و `rotateX` يدوران العنصر حول محوريهما (الأفقي والعمودي).
- `perspective` تضيف إحساسًا بالعمق، حيث تبدو الأجسام البعيدة أصغر والأقرب أكبر.
- يمكن دمج تحويلات 3D المتعددة في خاصية `transform` واحدة لتأثيرات بصرية معقدة وجذابة.

الانتقالات والتحويلات (Transitions and Transforms) — تطبيق الانتقالات والتحويلات لتأثيرات بصرية جذابة

تطبيق الانتقالات والتحويلات لتأثيرات بصرية جذابة

في CSS، تساعدنا الانتقالات (Transitions) والتحويلات (Transforms) في جعل عناصر صفحات الويب تتفاعل بشكل حيوي وجذاب. إنها تضيف حركة وتغييرات بصرية ناعمة دون الحاجة إلى JavaScript المعقد.

الانتقالات (Transitions)

تُستخدم الانتقالات لإحداث تغييرات تدريجية وناعمة بين حالات مختلفة لعنصر ما. بدلاً من تغيير خاصية CSS بشكل فوري، تجعل الانتقالات هذا التغيير يحدث ببطء خلال فترة زمنية محددة.

التحويلات (Transforms)

تسمح لك التحويلات بتعديل شكل أو حجم أو موضع عنصر في صفحة الويب في بعدين (2D) أو ثلاثة أبعاد (3D) دون التأثير على تدفق المستند. يمكنك تدوير العناصر، تغيير حجمها، نقلها، أو ميلها.

مثال عملي

```
.element-box {  
  width: 120px;  
  height: 80px;  
  background-color: #007bff;  
  border-radius: 5px;  
  margin: 20px;  
  display: flex;  
  justify-content: center;  
  align-items: center;
```

```
        color: white;
        font-weight: bold;
        cursor: pointer;
        transition: transform 0.4s ease-in-out;
    }

    .element-box:hover {
        transform: scale(1.1) rotate(5deg);
    }
```

شرح الكود

لدينا هنا مربع بسيط (.element-box). الخاصية `transition: transform 0.4s ease-in-out`; تخبر المتصفح أن أي تغيير يطرأ على خاصية `transform` يجب أن يتم بشكل ناعم خلال 0.4 ثانية. عندما يمر المؤشر فوق المربع (.element-box:hover)، يتم تطبيق `transform: scale(1.1) rotate(5deg)`; الذي يقوم بتكبير المربع بنسبة 10% وتدويره 5 درجات، ويتم هذا التغيير بسلسلة بفضل الانتقال الذي حددناه مسبقاً.

- **الانتقالات (Transitions):** تجعل التغييرات في خصائص CSS تحدث بشكل تدريجي وسلس.
- **التحويلات (Transforms):** تسمح لك بتعديل موضع وحجم وتدوير وميل العناصر بصرياً.
- **الدمج:** يمكن دمج الانتقالات والتحويلات لإنشاء تأثيرات بصرية تفاعلية وجذابة جداً.

المتغيرات والدوال في CSS — المتغيرات المخصصة (Custom Properties / CSS) (Variables) وكيفية تعريفها

المتغيرات المخصصة (Custom Properties / CSS) (Variables) في CSS

تُقدم CSS المتغيرات المخصصة، والتي تُعرف أيضاً بـ "متغيرات CSS"، كأداة قوية لتخزين القيم القابلة لإعادة الاستخدام في ورقة الأنماط الخاصة بك. هذا يسهل إدارة الألوان والأحجام وغيرها من الخصائص، مما يجعل كود CSS أكثر مرونة وأسهل في التعديل.

ما هي المتغيرات المخصصة؟

هي قيم تحددها أنت وتستطيع إعادة استخدامها في أماكن مختلفة من ورقة الأنماط. يتم تعريفها باستخدام بادئة `--` وتُستخدم باستدعاء دالة `var()`، مما يقلل التكرار ويحسن قابلية الصيانة.

مثال عملي

```
:root {
  --primary-color: #007bff;
  --text-color: #333;
  --base-font-size: 16px;
}

h1 {
  color: var(--primary-color);
  font-size: calc(var(--base-font-size) * 2.5);
}

p {
```

```
color: var(--text-color);
font-size: var(--base-font-size);
line-height: 1.6;
}
```

شرح الكود

في هذا المثال، قمنا بتعريف ثلاثة متغيرات مخصصة داخل العنصر `root`، وهو المحدد الذي يمثل العنصر الجذر للمستند (عادةً `<html>`). هذا يجعل المتغيرات متاحة عالمياً. ثم استخدمنا هذه المتغيرات داخل قواعد `h1` و `p` لتطبيق الألوان وحجم الخط. لاحظ كيف أن تغيير قيمة المتغير في مكان واحد (في `root`) سيؤثر على جميع الأماكن التي تستخدم هذا المتغير.

- **التعريف:** تبدأ المتغيرات المخصصة دائماً بـ `--`، مثل `--primary-color`.
- **الاستخدام:** لاستدعاء قيمة متغير، نستخدم الدالة `var(--variable-name)`.
- **المرونة:** تزيد المتغيرات من مرونة CSS وتسهل التعديلات المستقبلية على الأنماط.

المتغيرات والدوال في CSS — استخدام المتغيرات (var()) وتحديد نطاقها (Scope)

استخدام المتغيرات (var()) وتحديد نطاقها (Scope)

تُعدّ متغيرات CSS (التي تسمى أيضًا الخصائص المخصصة) أداة قوية تسمح لك بتعريف قيم يمكن إعادة استخدامها في ورقة الأنماط الخاصة بك. هذا يقلل من تكرار الكود، ويجعل التحديثات أسهل وأسرع، وينظم أنماطك بشكل فعال.

ما هي متغيرات CSS؟

متغيرات CSS هي قيم مخصصة تقوم أنت بتعريفها، وتبدأ دائمًا بعلامتين ناقص (--)، مثل --main-color. يمكنك استخدام هذه المتغيرات في أي مكان في CSS عن طريق الدالة var()، مثل color: var(--main-color);. يحدد نطاق المتغير (Scope) أين يمكن الوصول إليه، فالمتغيرات المعرّفة في root: تكون عالمية، بينما المعرّفة داخل محدد تكون محلية.

مثال عملي

```
:root {
  --primary-color: #007bff;
  --base-font-size: 16px;
}

.card {
  --card-bg: #f8f9fa; /* متغير محلي */
  background-color: var(--card-bg);
  border: 1px solid var(--primary-color);
  padding: 20px;
  font-size: var(--base-font-size);
  color: var(--primary-color);
}
```



```
.card h2 {  
  color: var(--primary-color);  
}
```

شرح الكود

في هذا المثال، قمنا بتعريف متغيرين عالميين (`base-font-size--` و `primary-color--`) داخل المحدد `root`، مما يجعلهما متاحين للاستخدام في جميع أنحاء ورقة الأنماط. ثم عرّفنا متغيرًا محليًا (`card-bg--`) داخل الفئة `card`، وهذا المتغير متاح فقط ضمن هذه الفئة وعناصرها الفرعية. نستخدم بعد ذلك الدالة `var()` لاستدعاء هذه المتغيرات وتطبيق قيمها على خصائص CSS المختلفة، مما يوضح كيفية عمل المتغيرات العالمية والمحلية.

الخلاصة

- **تقليل التكرار:** تسمح المتغيرات بكتابة كود CSS أنظف وأكثر قابلية للصيانة عن طريق تخزين القيم المتكررة.
- **سهولة التحديث:** تغيير قيمة متغير واحد في مكان واحد يؤثر على جميع الأماكن التي تستخدم هذا المتغير، مما يسرّع تحديث التصميم.
- **فهم النطاق:** من الضروري معرفة نطاق المتغير (عالمي أو محلي) لضمان تطبيق الأنماط بشكل صحيح وتجنب الأخطاء.

المتغيرات والدوال في CSS — الدوال

الشائعة في CSS مثل calc()

الدوال الشائعة في CSS: دالة calc()

في عالم CSS، الدوال (Functions) هي أدوات قوية يمكننا من إجراء عمليات ديناميكية أو تحويلات على القيم. دالة calc() هي إحدى هذه الدوال الأساسية والمفيدة للغاية، حيث تسمح لنا بإجراء عمليات حسابية بسيطة مباشرة داخل خصائص CSS.

مفهوم دالة calc()

تُستخدم دالة calc() لإجراء العمليات الحسابية الأساسية مثل الجمع (+)، الطرح (-)، الضرب (*)، والقسمة (/) لتعيين قيم خصائص CSS. يمكن لهذه الدالة أن تجمع بين وحدات قياس مختلفة (مثل % و px و em) في نفس التعبير، مما يوفر مرونة كبيرة في التصميم المتجاوب.

مثال عملي

```
.header {
  height: 60px;
}

.main-content {
  width: calc(100% - 40px);
  margin: 0 auto;
  min-height: calc(100vh - 60px); /* ارتفاع الشاشة بالكامل - ارتفاع header */
  padding: 20px;
  background-color: #f8f8f8;
}
```

شرح الكود

في هذا المثال، حددنا ارتفاع العنصر `header` ليكون 60 بكسل. ثم، استخدمنا `calc` () للعنصر `main-content` لتعيين عرضه ليكون 100% من عرض الحاوية الأب ناقص 40 بكسل (لتوفير مساحة على الجانبين). الأهم، استخدمنا `calc` () لتعيين الحد الأدنى لارتفاع المحتوى ليكون ارتفاع نافذة العرض بالكامل (100vh) مطروحاً منه ارتفاع الـ `header`، مما يضمن أن المحتوى يملأ المساحة المتبقية بشكل ديناميكي.

نقاط أساسية

- تسمح دالة `calc` () بتنفيذ العمليات الحسابية (+, -, *, /) على قيم خصائص CSS.
- يمكنها دمج وحدات قياس مختلفة (مثل %, vh, px) في تعبير حسابي واحد.
- تُعد أداة قوية جداً لإنشاء تصميمات ويب متجاوبة ومرنة دون الحاجة إلى استخدام JavaScript.

المتغيرات والدوال في CSS — تنظيم الأنماط وإعادة استخدامها باستخدام المتغيرات

تنظيم الأنماط وإعادة استخدامها باستخدام المتغيرات في CSS

تُعد المتغيرات في CSS أداة قوية تسمح لك بتعريف قيم قابلة لإعادة الاستخدام مرة واحدة، ثم استخدامها في أي مكان آخر في ملف الأنماط الخاص بك. هذا يجعل شيفرة CSS أكثر تنظيماً، وأسهل في القراءة، وأسرع في التعديل.

مفهوم المتغيرات في CSS

المتغيرات (CSS Custom Properties) هي خصائص مخصصة تبدأ بـ `--` تتبعها قيمة. تُعرّف هذه المتغيرات عادةً داخل محدد مثل `root:` لجعلها متاحة عالمياً، وتُستخدم باستخدام دالة `var()` لاستدعاء قيمتها.

مثال على الكود

```
:root {
  --primary-color: #007bff;
  --text-color: #333;
  --font-size-base: 16px;
}

body {
  font-family: sans-serif;
  color: var(--text-color);
  font-size: var(--font-size-base);
}

h1 {
  color: var(--primary-color);
}
```

```
font-size: calc(var(--font-size-base) * 2);
}

button {
  background-color: var(--primary-color);
  color: white;
  padding: 10px 15px;
  border: none;
  cursor: pointer;
}
```

شرح الكود

في هذا المثال، قمنا بتعريف ثلاثة متغيرات عالمية داخل محدد `root: --primary-color, --:` ، `font-size-base--` و `text-color`. بعد ذلك، استخدمنا هذه المتغيرات في عناصر مختلفة مثل `h1`، `body`، و `button` باستخدام `var()`. لاحظ كيف أن تغيير قيمة متغير واحد في `root:` سيحدث اللون الأساسي أو حجم الخط في جميع الأماكن التي تستخدم هذا المتغير.

- **تحسين التنظيم:** تجعل الشيفرة أكثر قابلية للقراءة والفهم من خلال تجميع القيم المتكررة في مكان واحد.
- **تعزيز الاتساق:** تساعد في الحفاظ على تصميم موحد للموقع بأكمله.
- **سهولة التعديل:** تسمح بتحديث التصميم بسرعة عن طريق تغيير قيمة المتغير في موقع واحد فقط.

بناء مشاريع بسيطة وتطبيق المفاهيم — تطبيق المفاهيم: بناء صفحة هبوط بسيطة (Landing Page)

تطبيق المفاهيم: بناء صفحة هبوط بسيطة (Landing Page)

في هذا الدرس، سنتعلم كيفية تجميع المفاهيم الأساسية في CSS لبناء صفحة هبوط (Landing Page) بسيطة وفعالة. سنركز على تطبيق التنسيقات لإنشاء تصميم متكامل يخدم غرضًا محددًا، مما يوضح كيف يمكن لـ CSS أن يحول البنية الخام إلى واجهة جذابة وعملية.

المفهوم الأساسي: تصميم صفحات الهبوط باستخدام CSS

صفحة الهبوط هي صفحة ويب مصممة لغرض محدد، مثل عرض منتج أو خدمة، أو جمع بيانات العملاء. يتطلب تصميمها استخدامًا ذكيًا لخصائص CSS لتنظيم المحتوى، تحديد الألوان والخطوط، وإنشاء تجربة مستخدم واضحة ومباشرة تحت الزائر على اتخاذ إجراء.

مثال على الكود

```
.hero-section {  
  background-color: #3498db;  
  color: white;  
  padding: 80px 20px;  
  text-align: center;  
  font-family: 'Arial', sans-serif;  
}  
  
.hero-section h1 {  
  font-size: 3em;  
  margin-bottom: 15px;  
}
```

```
.hero-section p {  
  font-size: 1.2em;  
  margin-bottom: 30px;  
}
```

شرح الكود

يوضح هذا الكود كيفية تنسيق قسم "البطل" (hero section)، وهو الجزء العلوي البارز في صفحة الهبوط. يقوم بتعيين لون خلفية أزرق جذاب ولون نص أبيض، مع إضافة مساحة داخلية (padding) كبيرة لجعل المحتوى يظهر بشكل واضح. كما يقوم بمحاذاة النص للمنتصف ويحدد عائلة الخط. بعد ذلك، يضبط الكود حجم الخط وعناصر التباعد (margin) للعناوين الرئيسية والفقرات داخل هذا القسم، لضمان مظهر احترافي وسهل القراءة.

- **التصميم الهرمي:** استخدم خصائص CSS لترتيب العناصر بشكل منطقي وموجه للهدف.
- **الجاذبية البصرية:** طبق الألوان والخطوط والتباعدات لإنشاء تصميم مريح للعين وجذاب.
- **التنسيق الموحد:** حافظ على اتساق التنسيقات عبر جميع أقسام الصفحة للحصول على مظهر احترافي.

بناء مشاريع بسيطة وتطبيق المفاهيم — تصميم بطاقات المنتج أو الخدمات (Product/Service Cards)

تصميم بطاقات المنتج أو الخدمات (Product/Service) (Cards)

تُعد بطاقات المنتج أو الخدمة (Product/Service Cards) من العناصر المرئية الأساسية في تصميم واجهات المستخدم، فهي تُمكننا من عرض المعلومات الهامة مثل اسم المنتج، سعره، ووصفه المختصر بطريقة جذابة ومنظمة. في هذا الدرس، سنتعلم كيف نستخدم CSS لبناء وتنسيق هذه البطاقات بفعالية.

المفهوم الأساسي: هيكلية وتصميم البطاقة

تعتمد بطاقات المنتج على هيكل HTML بسيط يتضمن حاوية رئيسية (مثل `div`) وعناصر داخلية (مثل `h3` للعنوان، `p` للوصف). نستخدم CSS لتطبيق الحدود، الظلال، الهوامش، والألوان لمنح البطاقة مظهرها النهائي الجذاب والواضح.

مثال على الكود

```
.product-card {  
  border: 1px solid #ddd;  
  border-radius: 10px;  
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);  
  padding: 20px;  
  margin: 15px;  
  width: 280px;  
  text-align: center;  
  background-color: #fff;  
}
```



```
.product-card h3 {  
    color: #333;  
    margin-bottom: 10px;  
}
```

شرح الكود

يحدد هذا الكود تنسيق البطاقة الأساسي `product-card`. `border` يضيف إطارًا رقيقًا حول البطاقة، و `border-radius` يجعل حوافها مستديرة قليلًا. لتعزيز مظهرها، يضيف `box-shadow` ظلًا خفيفًا، و `padding` يترك مسافة داخلية بين المحتوى والحواف. `margin` يضمن وجود مسافة خارجية تفصل البطاقة عن العناصر الأخرى. `width` يحدد عرض البطاقة، و `text-align: center` يوسط المحتوى النصي. وأخيرًا، `background-color` يعطي البطاقة لون خلفية. أما `product-card h3`، فيحدد لون عنوان المنتج ويهامشه السفلي.

نقاط أساسية

- **التنظيم البصري:** تساعد البطاقات على عرض المعلومات بطريقة منظمة وسهلة الاستيعاب.
- **جاذبية التصميم:** استخدام خصائص CSS مثل `border-radius` و `box-shadow` يعزز المظهر الجمالي للبطاقة.
- **مرونة التطبيق:** يمكن تكييف تصميم البطاقات ليتناسب مع أنواع مختلفة من المنتجات أو الخدمات ودمجها بسهولة في أي مشروع.

بناء مشاريع بسيطة وتطبيق المفاهيم — بناء قائمة تنقل (Navigation Menu) متجاوبة

بناء قائمة تنقل (Navigation Menu) متجاوبة

تعد قائمة التنقل جزءًا أساسيًا في أي موقع ويب، فهي تساعد الزوار على استكشاف المحتوى بسهولة. في هذا الدرس، سنتعلم كيف نصمم قائمة تنقل جميلة وعملية تكون **متجاوبة**، أي تتكيف تلقائيًا مع أحجام الشاشات المختلفة باستخدام CSS.

مفتاح المفهوم: القائمة المتجاوبة

القائمة المتجاوبة هي قائمة يتغير تصميمها وشكلها تلقائيًا بناءً على حجم شاشة الجهاز الذي يتصفح الموقع. الهدف هو توفير تجربة مستخدم ممتازة وسهلة الاستخدام على الهواتف الذكية والأجهزة اللوحية وأجهزة الكمبيوتر المكتبية على حد سواء.

مثال الكود

```
nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
  background-color: #333;
  display: flex; /* لعرض العناصر أفقيًا */
  justify-content: space-around;
}

nav li a {
  display: block;
  color: white;
  text-align: center;
  padding: 14px 16px;
  text-decoration: none;
}
```

```

        nav li a:hover {
            background-color: #575757;
        }

        /* جزء التجاوب: عند الشاشات الصغيرة */
        @media (max-width: 600px) {
            nav ul {
                flex-direction: column; /* لعرض العناصر عموديًا */
            }
            nav li {
                text-align: center;
            }
        }
    }
}

```

شرح الكود

يبدأ الكود بتصميم القائمة الأساسية `nav ul` بإزالة النقاط وإعطائها خلفية داكنة. استخدمنا `display: flex` لجعل عناصر القائمة (الروابط) تظهر جنبًا إلى جنب بشكل أفقي. ثم يتم تنسيق الروابط `nav li a` بجعلها كتلًا قابلة للنقر وتحديد لون الخط وإزالة التسطير.

الجزء المهم للتجاوب هو `@media (max-width: 600px)`. هذا يعني أنه عندما تكون الشاشة أصغر من 600 بكسل، تتغير اتجاه عناصر القائمة `flex-direction: column` لتصبح فوق بعضها عموديًا، مما يجعلها مناسبة للهواتف والشاشات الصغيرة.

- **أهمية قائمة التنقل:** تسهل على المستخدمين التنقل في الموقع واكتشاف المحتوى.
- **خاصية `display: flex`:** أداة قوية وفعالة لتصميم القوائم الأفقية والمرنة.
- **استعلامات الوسائط (Media Queries):** ضرورية جدًا لجعل التصميم متجاوبًا ويتكيف مع مختلف أحجام الشاشات والأجهزة.

بناء مشاريع بسيطة وتطبيق المفاهيم — مراجعة أفضل الممارسات في كتابة CSS

مراجعة أفضل الممارسات في كتابة CSS

في هذا الدرس، نراجع أهم الممارسات التي تضمن لك كتابة أكواد CSS نظيفة، سهلة القراءة، قابلة للصيانة، وفعالة. تطبيق هذه الممارسات يساعدك في بناء مشاريع مستقرة ويسهل العمل عليها في المستقبل، سواء كنت تعمل بمفردك أو ضمن فريق.

كتابة CSS نظيف ومنظم

يعني هذا استخدام أسماء فئات (classes) واضحة وذات معنى، تنظيم الأكواد في ملفات منطقية، وتجنب التكرار. الهدف هو جعل أكوادك سهلة الفهم والتعديل لأي شخص يقرأها، بمن فيهم أنت نفسك بعد فترة طويلة.

مثال على الكود

```
.main-header {  
  background-color: #333;  
  color: white;  
  padding: 15px;  
  text-align: center;  
}  
  
.btn-primary {  
  background-color: #007bff;  
  color: white;  
  padding: 10px 15px;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
}
```

```
.text-center {  
  text-align: center;  
}
```

شرح الكود

توضح هذه الأمثلة استخدام أسماء فئات (classes) واضحة وذات معنى. فالفئة `main-header` تصف بوضوح وظيفتها كعنوان رئيسي. والفئة `btn-primary` تشير إلى زر أساسي. بينما `text-center` هي فئة مساعدة لوضع النص في المنتصف. هذه التسميات تجعل الكود سهل الفهم والاستخدام.

- **التسمية الواضحة:** استخدم أسماء فئات (classes) معرفة بوضوح لوظيفتها، مثل `card-title` بدلاً من `ct`.
- **التنظيم الجيد:** قسّم أكوادك إلى كتل منطقية أو ملفات منفصلة لتسهيل إدارتها وقراءتها.
- **تجنب التكرار:** حاول كتابة قواعد CSS عامة يمكن إعادة استخدامها، مثل الفئات المساعدة (utility classes).